

# CORE 05 — Computer Networks

Shared-core file 5 of the NPSC CTSE 2026 prep set.

Three depths for every topic: **one-line fact** (MCQ), **5-mark answer** (~5–6 points),

**10/15-mark answer** (worked explanation + diagram).

## 0. Why this matters — where networks appear in your three papers

| Elective          | Where                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | How heavy  |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| Computer Forensic | TP-I §7 — a full unit, and the most detailed of the three. Names OSI, TCP/IP, IP & MAC address, channel capacity, all transmission media, multiplexing, switching, ISDN, ATM, internetworking devices, concatenated virtual circuits, tunnelling, fragmentation, firewalls, routing algorithms, network security, public/secret key cryptography, DNS, resource records, name servers, e-mail architecture, WWW. Plus TP-II §2 (email protocols, header analysis) and TP-II §1 (packet sniffing, spoofing, web security). | Very heavy |
| CS Diploma        | P-I §5 (network concept, transmission modes, media, topologies, architectures, connectivity devices, <b>all 7 OSI layers</b> , TCP/IP, NetBEUI, IPX/SPX, IEEE standards) and P-I §6 (Internet technology, ISP, dial-up/leased/ISDN/VSAT, WWW, email, FTP, Telnet).                                                                                                                                                                                                                                                        | Heavy      |
| CS Degree         | P-II §4 — "Basic Network Concepts, Network Topologies, Networking devices, Transmission media, OSI Model, TCP/IP fundamentals, Knowledge of Internet, Internet routing, IP address."                                                                                                                                                                                                                                                                                                                                      | Heavy      |

**This is the single best-shared topic in the whole syllabus.** All three electives name OSI, TCP/IP, IP addressing, media, topologies and devices. Study it once, answer it three times. Budget accordingly — this file deserves more of your 3–4 hours a day than almost anything else.

### Topic checklist

- [ ] What a network is; goals and applications
- [ ] Classification by scale: PAN, LAN, MAN, WAN, internetwork; wireless variants
- [ ] Transmission modes: simplex, half-duplex, full-duplex
- [ ] Transmission media: UTP/STP, coaxial, fibre; radio, microwave, infrared, satellite
- [ ] Topologies: bus, star, ring, mesh, tree, hybrid — with advantages/disadvantages
- [ ] Architectures: peer-to-peer vs client-server
- [ ] Connectivity devices mapped to OSI layers
- [ ] **OSI 7-layer model — every layer's function, PDU, protocols, devices**
- [ ] TCP/IP model and OSI-vs-TCP/IP comparison
- [ ] **IP addressing: classes, subnet masks, subnetting numericals, CIDR, VLSM, private ranges**
- [ ] IPv4 vs IPv6; NAT; DHCP
- [ ] MAC addresses; ARP and RARP
- [ ] Routing: static/dynamic, distance vector vs link state, RIP/OSPF/BGP
- [ ] TCP vs UDP; three-way handshake; ports; flow and congestion control
- [ ] DNS: hierarchy, resource records, name servers, resolution
- [ ] Email: architecture, SMTP, POP3, IMAP, MIME, header analysis
- [ ] HTTP, HTTPS, WWW, URL structure
- [ ] Switching: circuit, message, packet (datagram vs virtual circuit)
- [ ] Multiplexing: FDM, TDM (sync/stat), WDM
- [ ] **Channel capacity: Nyquist and Shannon numericals**
- [ ] ISDN (narrowband/broadband), ATM, leased lines, VSAT

- [ ] Network security: threats, cryptography, digital signatures, firewalls
- [ ] Tunnelling, fragmentation, concatenated virtual circuits
- [ ] IEEE 802 standards; Ethernet; CSMA/CD and CSMA/CA

# 1. Fundamentals and classification

## Concept

A **computer network** is a collection of autonomous computing devices interconnected by a transmission medium so that they can exchange data and share resources. The word **autonomous** carries a mark: if one machine can forcibly start, stop or control another, that is a **master–slave** or **distributed system**, not a network. Each node must be an independent computer in its own right.

Why build one at all? Four classic goals, worth listing:

| Goal                                | Explanation                                                                               |
|-------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Resource sharing</b>             | Printers, storage, applications, databases available to everyone regardless of location   |
| <b>Communication</b>                | Email, chat, video conferencing, VoIP                                                     |
| <b>Reliability</b>                  | Replicated files and alternate paths — if one machine or link dies, another serves        |
| <b>Cost reduction / scalability</b> | Many small PCs sharing a server is cheaper than one mainframe; add capacity incrementally |

Two more usually added: **centralised management/security** (one point of backup and access control) and **load sharing / distributed processing**.

## Classification by geographical scale

| Type       | Full form                 | Span                                               | Owned by                                 | Speed                               | Media                                         | Example                                                          |
|------------|---------------------------|----------------------------------------------------|------------------------------------------|-------------------------------------|-----------------------------------------------|------------------------------------------------------------------|
| <b>PAN</b> | Personal Area Network     | ~1–10 m (around a person)                          | Individual                               | Low (up to a few Mbps)              | Bluetooth, IrDA, USB, Zigbee, NFC             | Phone + earbuds + smartwatch                                     |
| <b>LAN</b> | Local Area Network        | <b>A room, building or campus</b> (up to ~1–10 km) | <b>Private / single organisation</b>     | <b>High: 10 Mbps – 10 Gbps</b>      | Twisted pair, fibre, Wi-Fi                    | Office/college network                                           |
| <b>CAN</b> | Campus Area Network       | Several adjacent buildings                         | Single organisation                      | High                                | Fibre                                         | University campus                                                |
| <b>MAN</b> | Metropolitan Area Network | <b>A city (~10–100 km)</b>                         | Often a service provider or a consortium | Moderate–high                       | Fibre, microwave                              | Cable TV network, city-wide Wi-Fi, SONET ring, DQDB (IEEE 802.6) |
| <b>WAN</b> | Wide Area Network         | <b>Country, continent, worldwide</b>               | Usually <b>leased from a carrier</b>     | <b>Lower per link, high latency</b> | Leased lines, satellite, fibre backbone, PSTN | The Internet, bank branch network                                |

|                     |                     |                                                        |          |   |           |                                                |
|---------------------|---------------------|--------------------------------------------------------|----------|---|-----------|------------------------------------------------|
| <b>GAN</b>          | Global Area Network | Worldwide, incl. satellite                             | —        | — | Satellite | Global mobile roaming                          |
| <b>Internetwork</b> | —                   | Two or more distinct networks joined by <b>routers</b> | Multiple | — | Mixed     | <b>The Internet</b> — the largest internetwork |

**The three numbers examiners test:** LAN has the **highest** data rate and the **lowest** error rate and delay; WAN has the **largest** span and the **highest** delay; MAN sits between the two. Ownership is the other discriminator: a LAN is privately owned; a WAN's links are almost always leased from a public carrier.

**Wireless networks** — classify by the same scale: **WPAN** (Bluetooth, IEEE 802.15), **WLAN** (Wi-Fi, IEEE 802.11), **WMAN** (WiMAX, IEEE 802.16), **WWAN** (cellular — GSM/3G/4G LTE/5G). Their common characteristics: no cabling cost, mobility, easy expansion — but lower security (the medium is public air), interference, and lower throughput than wired equivalents.

**Other classifications** worth a line each:

- By **transmission technology**: **broadcast** networks (one shared channel, every node hears every packet; typical of LANs) vs **point-to-point** networks (dedicated links between pairs; typical of WANs, requiring routing).
- By **relationship**: peer-to-peer vs client-server (see §5).
- By **connectivity**: intranet (private, organisation-internal), extranet (intranet extended to selected partners), Internet (public, global).

## Likely exam questions

- **[5]** Define a computer network. State any five goals/advantages of networking.
- **[5]** Differentiate between LAN, MAN and WAN in tabular form.
- **[5]** What is an internetwork? How does it differ from a LAN?
- **[10]** Explain the classification of computer networks based on geographical area, giving span, ownership, speed, media and one example for each type.

## MCQ traps

| Trap                               | Correct answer                                                         |
|------------------------------------|------------------------------------------------------------------------|
| "Which has the highest data rate?" | <b>LAN</b> .                                                           |
| "Which spans a city?"              | <b>MAN</b> .                                                           |
| "The Internet is a?"               | <b>WAN / internetwork</b> , not a LAN.                                 |
| "IEEE standard for MAN (DQDB)?"    | <b>802.6</b> . Wi-Fi = 802.11, Ethernet = 802.3, Bluetooth = 802.15.1. |
| "Bluetooth range network?"         | <b>PAN</b> .                                                           |
| "Nodes in a network must be?"      | <b>Autonomous</b> .                                                    |

## 2. Transmission modes

### Concept

Transmission mode (or "communication mode") describes the **direction of signal flow** between two linked devices. Do not confuse it with serial vs parallel transmission — that is about *how many bits at a time*, which is a different question.

| Mode                        | Direction                                                                                     | Analogy                             | Channel capacity use                                                                    | Examples                                                                               |
|-----------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>Simplex</b>              | <b>One way only.</b> One device is permanently the sender, the other permanently the receiver | Radio broadcast, a lecture          | Entire capacity used in the one direction                                               | Keyboard → CPU, CPU → monitor, TV/radio broadcast, loudspeaker, sensor telemetry       |
| <b>Half-duplex</b>          | <b>Both ways, but only one at a time.</b> The link must be "turned around"                    | Walkie-talkie, a single-lane bridge | Entire capacity used by whichever side is transmitting                                  | Walkie-talkie, CB radio, <b>classic Ethernet on a hub (with CSMA/CD)</b> , older Wi-Fi |
| <b>Full-duplex (duplex)</b> | <b>Both ways simultaneously</b>                                                               | Telephone conversation              | Capacity <b>shared</b> between the two directions (or two separate physical paths used) | Telephone, modern <b>switched Ethernet</b> , mobile phone call                         |

**How full-duplex is achieved** — a mark-worthy detail: either the link has **two physically separate transmission paths** (one per direction, e.g. two pairs in a UTP cable), or the **capacity of a single channel is divided** between the two directions (by frequency, as in FDM). Say which one you mean.

Draw three small diagrams: one arrow → for simplex; two arrows on the same line with a "one at a time" note for half-duplex; two parallel arrows in opposite directions for full-duplex.

## MCQ traps

| Trap                                 | Correct answer                                                                              |
|--------------------------------------|---------------------------------------------------------------------------------------------|
| "Walkie-talkie is?"                  | <b>Half-duplex</b> .                                                                        |
| "Keyboard to computer is?"           | <b>Simplex</b> .                                                                            |
| "Telephone is?"                      | <b>Full-duplex</b> .                                                                        |
| "Ethernet on a hub?"                 | <b>Half-duplex</b> (collisions possible). On a <b>switch</b> , full-duplex (no collisions). |
| "Simplex vs half-duplex difference?" | Simplex can <b>never</b> reverse direction.                                                 |

## 3. Transmission media

### Concept

Transmission media split cleanly in two:

- **Guided (wired / bounded)** — the signal is confined inside a physical conductor: twisted pair, coaxial, fibre optic. The medium itself determines the capacity and the distance.
- **Unguided (wireless / unbounded)** — the signal is broadcast through air/vacuum: radio, microwave, infrared, satellite. Here the *frequency band* and the antenna determine behaviour.

### 3.1 Twisted pair

Two insulated copper wires **twisted together in a helix**. Why twisted? Because if the wires ran parallel, the one nearer an interference source would pick up more noise than the other. Twisting makes both wires experience the *same* interference on average, so a differential

receiver subtracts it out. More twists per unit length = better noise rejection. **State the reason for the twist** — it is a standard 2-mark point.

|                 | UTP (Unshielded Twisted Pair)                                  | STP (Shielded Twisted Pair)                                                     |
|-----------------|----------------------------------------------------------------|---------------------------------------------------------------------------------|
| Shielding       | None beyond the plastic jacket                                 | Metal foil/braid <b>around each pair and/or the whole bundle</b>                |
| Noise immunity  | Lower; relies purely on twisting                               | <b>Higher</b> — the shield blocks EMI and reduces crosstalk                     |
| Cost            | <b>Cheapest</b> medium                                         | More expensive                                                                  |
| Diameter/weight | Thin, light, very flexible                                     | Thicker, heavier, harder to install                                             |
| Grounding       | Not required                                                   | <b>Shield must be grounded.</b> or it acts as an antenna and makes things worse |
| Connector       | <b>RJ-45</b> (8 pins) for data; RJ-11 (4/6 pins) for telephone | RJ-45 / STP connectors                                                          |
| Typical use     | <b>The dominant LAN cable</b> ; telephone local loop           | Industrial/noisy environments, Token Ring                                       |

**UTP categories** — memorise Cat 5e and Cat 6:

| Category      | Bandwidth | Data rate                | Use                                     |
|---------------|-----------|--------------------------|-----------------------------------------|
| Cat 1         | —         | Voice only               | Old telephone                           |
| Cat 3         | 16 MHz    | 10 Mbps                  | 10BASE-T                                |
| Cat 5         | 100 MHz   | 100 Mbps                 | Fast Ethernet                           |
| <b>Cat 5e</b> | 100 MHz   | <b>1 Gbps</b>            | Gigabit Ethernet — the common workhorse |
| <b>Cat 6</b>  | 250 MHz   | 1 Gbps (10 Gbps to 55 m) | Modern installations                    |
| Cat 6a        | 500 MHz   | 10 Gbps to 100 m         | Data centres                            |
| Cat 7         | 600 MHz   | 10 Gbps                  | Shielded                                |

Maximum UTP segment length for Ethernet: **100 metres**. That is an MCQ. Ethernet naming reads as *speed–signalling–medium*: **10BASE-T** = 10 Mbps, baseband, twisted pair; **100BASE-TX** = Fast Ethernet; **1000BASE-T** = Gigabit; **10BASE5** = 10 Mbps baseband 500 m (thick coax, "thicknet"); **10BASE2** = 185 m (thin coax, "thinnet").

Straight-through cable (both ends T568B) connects **unlike** devices (PC↔switch); **crossover** cable connects **like** devices (PC↔PC, switch↔switch). Modern ports auto-sense (Auto-MDIX), but the exam still asks.

## 3.2 Coaxial cable

A **central copper conductor**, surrounded by an **insulating dielectric**, then a **braided metallic shield** (which is also the second conductor and the ground), then the outer plastic **jacket**. The shield completely encircles the core, which is why coax carries much higher frequencies than twisted pair with far less radiation and interference.

Draw the cross-section: four concentric circles labelled core / insulator / braided shield / jacket. Easy diagram marks.

| Property       | Value                                                                                                                   |
|----------------|-------------------------------------------------------------------------------------------------------------------------|
| Bandwidth      | Up to ~750 MHz–1 GHz; typically 10–100 Mbps for data                                                                    |
| Distance       | 185 m (thinnet, RG-58) / 500 m (thicknet, RG-8) per segment                                                             |
| Noise immunity | <b>Better than twisted pair, worse than fibre</b>                                                                       |
| Cost           | More than UTP, far less than fibre                                                                                      |
| Connectors     | <b>BNC</b> (with T-connector and 50 $\Omega$ terminator) for LANs; <b>F-type</b> for cable TV                           |
| Impedance      | <b>50 <math>\Omega</math></b> for data (RG-58/RG-8); <b>75 <math>\Omega</math></b> for cable TV (RG-59/RG-6)            |
| Modes          | <b>Baseband</b> (single digital channel, whole bandwidth) and <b>broadband</b> (FDM, many analogue channels — cable TV) |
| Uses           | Legacy bus Ethernet, cable television, cable broadband, CCTV                                                            |

### 3.3 Fibre optic cable

Data travels as **pulses of light**, not electricity. A glass or plastic **core** is surrounded by **cladding** of *lower refractive index*; a ray entering at greater than the **critical angle** undergoes **total internal reflection** and is trapped in the core. Around them sits a protective **buffer/jacket**. **Total internal reflection is the principle** — say that phrase.

| Mode                          | Core diameter                        | Light source | Distance                      | Bandwidth                                                                        | Cost    |
|-------------------------------|--------------------------------------|--------------|-------------------------------|----------------------------------------------------------------------------------|---------|
| <b>Multimode step-index</b>   | 50–100 $\mu\text{m}$                 | LED          | Short                         | Lowest (high modal dispersion — rays take different-length paths, pulse spreads) | Low     |
| <b>Multimode graded-index</b> | 50–62.5 $\mu\text{m}$                | LED          | Medium (~2 km)                | Medium (refractive index varies gradually, equalising path times)                | Medium  |
| <b>Single-mode</b>            | <b>8–10 <math>\mu\text{m}</math></b> | <b>Laser</b> | <b>Very long (60–100+ km)</b> | <b>Highest</b> (only one path, negligible modal dispersion)                      | Highest |

| Advantages                                                        | Disadvantages                                                          |
|-------------------------------------------------------------------|------------------------------------------------------------------------|
| <b>Enormous bandwidth</b> (Tbps)                                  | <b>Expensive</b> cable, transceivers and test equipment                |
| <b>Complete immunity to EMI/RFI</b> — no electrical signal at all | <b>Fragile</b> — glass; sharp bends break it                           |
| <b>Very low attenuation</b> → long spans without repeaters        | <b>Installation and splicing need skilled labour</b> and special tools |
| <b>Very secure</b> — extremely hard to tap without detection      | <b>Unidirectional</b> — two fibres needed for full-duplex              |
| Light weight, small diameter                                      | Cannot carry power to remote equipment                                 |
| No crosstalk, no fire/spark hazard                                |                                                                        |

Connectors: **SC, ST, LC, MT-RJ, FC**.

### 3.4 Unguided (wireless) media

The electromagnetic spectrum from about 3 kHz to 900 THz is divided by frequency; each band

has characteristic propagation.

| Medium             | Frequency                                   | Propagation                                                                                                                                                                                                        | Key characteristics                                                                                                                                          | Uses                                                                                                          |
|--------------------|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Radio waves</b> | <b>3 kHz – 1 GHz</b>                        | <b>Omnidirectional</b> — travel in all directions; sky-wave/ground-wave; <b>penetrate walls</b>                                                                                                                    | Long distance, low frequency = low bandwidth. <b>Susceptible to interference</b> from any other source on the same band. Bands are <b>regulated/licensed</b> | AM/FM radio, television, cordless phones, paging, maritime radio                                              |
| <b>Microwaves</b>  | <b>1 GHz – 300 GHz</b>                      | <b>Unidirectional / line-of-sight</b> — focused by a <b>parabolic dish</b> ; <b>cannot penetrate walls well</b> ; blocked by hills and buildings; affected by rain (rain fade) at high frequency                   | Sending and receiving antennas must be <b>precisely aligned</b> . Repeater towers every <b>~50 km</b> because of earth curvature. High bandwidth             | Terrestrial microwave links, <b>satellite communication</b> , cellular backhaul, Wi-Fi (2.4/5 GHz), Bluetooth |
| <b>Infrared</b>    | <b>300 GHz – 400 THz</b>                    | Line-of-sight; <b>cannot pass through walls</b>                                                                                                                                                                    | That inability is a <b>feature</b> : no interference between adjacent rooms, and better security. Short range only. Blocked by sunlight                      | TV remote controls, IrDA device-to-device, wireless keyboards/mice                                            |
| <b>Satellite</b>   | Uplink/downlink microwave (C, Ku, Ka bands) | <b>GEO</b> at 35,786 km (appears stationary; <b>3 satellites cover the globe</b> ; ~250–280 ms one-way <b>propagation delay</b> ), MEO (5,000–20,000 km, GPS), LEO (500–2,000 km, low delay, needs constellations) | Huge coverage including remote/oceanic areas; <b>high latency for GEO</b> ; expensive; weather-affected                                                      | TV broadcast, VSAT, GPS, remote-area Internet, military                                                       |

**VSAT** (Very Small Aperture Terminal), named in your Diploma syllabus: a small (0.75–3.8 m) satellite dish at a remote site communicating via a GEO satellite with a central **hub** station, usually in a **star topology**. Used for banking/ATM networks, rural connectivity, lottery terminals — anywhere terrestrial lines are absent.

### 3.5 Comparison table — memorise this

| Criterion             | UTP                     | STP        | Coaxial       | Fibre optic             | Wireless                  |
|-----------------------|-------------------------|------------|---------------|-------------------------|---------------------------|
| <b>Signal carried</b> | Electrical              | Electrical | Electrical    | <b>Light</b>            | Electromagnetic wave      |
| <b>Bandwidth</b>      | Moderate (up to 1 Gbps) | Moderate   | High          | <b>Very high (Tbps)</b> | Low–moderate              |
| <b>Max distance</b>   | <b>100 m</b>            | 100 m      | 185 m / 500 m | <b>Tens of km</b>       | Varies (10 m – 1000s km)  |
| <b>Attenuation</b>    | High                    | High       | Moderate      | <b>Very low</b>         | High, varies with weather |
| <b>EMI immunity</b>   | <b>Poor</b>             | Good       | Good          | <b>Immune</b>           | Poor                      |

|                       |                |                  |                      |                           |                                     |
|-----------------------|----------------|------------------|----------------------|---------------------------|-------------------------------------|
| <b>Security (tap)</b> | Easy to tap    | Easy             | Easy                 | <b>Very difficult</b>     | <b>Easiest — broadcast in air</b>   |
| <b>Cost</b>           | <b>Lowest</b>  | Low–moderate     | Moderate             | <b>Highest</b>            | Low cable cost, high equipment cost |
| <b>Installation</b>   | <b>Easiest</b> | Moderate         | Moderate             | <b>Difficult, skilled</b> | Easy, no cabling                    |
| <b>Typical use</b>    | LAN, telephone | Noisy industrial | Cable TV, legacy LAN | Backbone, WAN, long haul  | Mobility, difficult terrain         |

## Likely exam questions

- **[5]** Differentiate between guided and unguided transmission media.
- **[5]** Explain twisted pair cable. Differentiate UTP and STP.
- **[5]** Why are the wires in a twisted pair twisted? Explain UTP categories.
- **[5]** Explain the construction and working principle of optical fibre.
- **[5]** State four advantages and three disadvantages of optical fibre over copper.
- **[5]** Differentiate between radio waves, microwaves and infrared.
- **[10]** Explain the various transmission media used in computer networks with diagrams, and compare them on bandwidth, distance, noise immunity, security and cost.
- **[15]** Discuss guided and unguided transmission media in detail. Explain the construction of coaxial and fibre optic cables with diagrams, describe single-mode and multimode fibre, and compare all media in a table.

## MCQ traps

| Trap                                     | Correct answer                                                |
|------------------------------------------|---------------------------------------------------------------|
| "Which medium is immune to EMI?"         | <b>Fibre optic</b> .                                          |
| "Principle of optical fibre?"            | <b>Total internal reflection</b> .                            |
| "Cladding has refractive index ...?"     | <b>Lower</b> than the core.                                   |
| "Which fibre uses a laser?"              | <b>Single-mode</b> . Multimode uses <b>LED</b> .              |
| "UTP maximum segment length?"            | <b>100 m</b> .                                                |
| "Connector for UTP?"                     | <b>RJ-45</b> (RJ-11 = telephone).                             |
| "Coax data impedance?"                   | <b>50 <math>\Omega</math></b> (75 $\Omega$ = cable TV).       |
| "Which wireless cannot penetrate walls?" | <b>Infrared</b> (and largely microwave); <b>radio</b> can.    |
| "GEO satellite altitude?"                | <b>35,786 km</b> (~36,000 km); propagation delay ~250–280 ms. |
| "Microwave needs?"                       | <b>Line of sight</b> . dish alignment.                        |
| "Cheapest medium?"                       | <b>UTP</b> .                                                  |

## 4. Network topologies

### Concept

**Topology** is the *arrangement* of nodes and links. Distinguish **physical topology** (how

the cables actually run) from **logical topology** (how the data actually flows). They can differ: modern Ethernet on a switch is **physically a star** but **logically** behaves like a point-to-point mesh; classic 10BASE-T on a **hub** is physically a star but **logically a bus** (a single collision domain). Making that distinction is worth a mark.

## 4.1 Bus topology

All devices connect to a **single shared backbone cable (the bus)**, with a **terminator** at each end to absorb the signal and prevent reflections. A transmission propagates in both directions and every node hears it; only the addressed node keeps it.

| Advantages                                      | Disadvantages                                                           |
|-------------------------------------------------|-------------------------------------------------------------------------|
| <b>Least cable</b> — cheapest to install        | <b>A break in the backbone brings down the whole network</b>            |
| Simple, easy to extend for small networks       | <b>Very hard to troubleshoot</b> — a fault anywhere affects everything  |
| Failure of a <b>node</b> does not affect others | <b>Collisions</b> — only one node may transmit at a time; needs CSMA/CD |
| No central device needed                        | Performance <b>degrades sharply</b> as nodes increase                   |
|                                                 | Limited cable length and node count; terminators required               |

## 4.2 Star topology

Every device has a **dedicated point-to-point link to a central device** (hub or switch). All traffic passes through the centre.

| Advantages                                                         | Disadvantages                                                               |
|--------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>Easy to install, reconfigure, add/remove nodes</b>              | <b>Central device is a single point of failure</b> — hub dies, network dies |
| <b>Easy fault isolation</b> — a bad link affects only one node     | <b>More cable</b> than bus (one run per node)                               |
| <b>Robust</b> — one cable break isolates only that node            | Cost of the central device                                                  |
| High performance with a <b>switch</b> (no collisions, full duplex) | Performance depends entirely on the central device's capacity               |
| <b>The dominant modern LAN topology</b>                            |                                                                             |

## 4.3 Ring topology

Each device connects to exactly two neighbours, forming a **closed loop**. Data travels in **one direction** (or both in a dual ring), and **each node acts as a repeater**, regenerating the signal before passing it on. Access is usually controlled by a **token** — only the holder of the token may transmit, so there are **no collisions**.

| Advantages                                                               | Disadvantages                                                                         |
|--------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>No collisions</b> — deterministic, predictable access time under load | <b>A single node or link failure breaks the entire ring</b> (unless dual-ring/bypass) |
| Signal is <b>regenerated at every node</b> → long distances possible     | <b>Adding or removing a node disrupts</b> the network                                 |
| Equal access for all nodes (fair)                                        | <b>Difficult to troubleshoot</b>                                                      |

Performs better than bus under heavy load

Data passes through many nodes → **higher latency**, and a privacy concern

Examples: **Token Ring (IEEE 802.5)**, **FDDI** (dual counter-rotating fibre ring, 100 Mbps), SONET rings.

## 4.4 Mesh topology

**Every node is connected to every other node** by a dedicated point-to-point link (full mesh).

**The formula you must know:** for **n** nodes,

- Number of links =  $n(n - 1)/2$  (duplex links)
- Ports per device =  $n - 1$

*Worked:* for  $n = 8 \rightarrow \text{links} = 8 \times 7 / 2 = \mathbf{28}$ ; each device needs **7** ports.

For  $n = 10 \rightarrow 45$  links, 9 ports each.

| Advantages                                                              | Disadvantages                                          |
|-------------------------------------------------------------------------|--------------------------------------------------------|
| <b>Highest reliability / fault tolerance</b> — many alternate paths     | <b>Enormous cabling and port cost</b> — grows as $n^2$ |
| <b>Dedicated links</b> → no sharing, no congestion, guaranteed capacity | <b>Very difficult to install and reconfigure</b>       |
| <b>Privacy and security</b> — traffic doesn't traverse other nodes      | Impractical beyond a small number of nodes             |
| <b>Easy fault identification and isolation</b>                          | Bulk of wiring exceeds available space                 |

A **partial mesh** connects only the critical nodes to everything — the practical compromise used in **WAN backbones and the Internet core**.

## 4.5 Tree (hierarchical) and hybrid

**Tree:** a hierarchy of star networks connected to a backbone — a "star of stars". Used in large buildings/campuses (core → distribution → access). Advantage: scalable, easy to segment and manage; disadvantage: failure of the backbone or a high-level node cuts off whole branches.

**Hybrid:** any combination — star-bus, star-ring. **The Internet is a hybrid topology**. Most real networks are hybrid. Advantage: flexible, scalable, inherits strengths; disadvantage: complex design and management, costly.

## 4.6 Comparison summary

| Topology    | Cable cost     | Reliability        | Ease of fault isolation | Scalability      | Collision risk | Central point of failure |
|-------------|----------------|--------------------|-------------------------|------------------|----------------|--------------------------|
| <b>Bus</b>  | Lowest         | <b>Poor</b>        | <b>Poor</b>             | Poor             | High           | Backbone                 |
| <b>Star</b> | Moderate       | Good               | <b>Excellent</b>        | <b>Excellent</b> | None (switch)  | <b>Hub/switch</b>        |
| <b>Ring</b> | Moderate       | Poor (single ring) | Poor                    | Poor             | None (token)   | Any node                 |
| <b>Mesh</b> | <b>Highest</b> | <b>Excellent</b>   | Excellent               | Poor             | None           | None                     |
| <b>Tree</b> | High           | Moderate           | Good                    | <b>Excellent</b> | Depends        | Root/backbone            |

## Likely exam questions

- **[5]** Define topology. Explain bus topology with a diagram, listing two advantages and two disadvantages.
- **[5]** Compare star and ring topologies.
- **[5]** In a mesh topology with 10 nodes, how many links and how many ports per device are required? Derive the formula.
- **[5]** Differentiate between physical and logical topology.
- **[10]** Explain the various network topologies with neat diagrams, stating the advantages and disadvantages of each, and state which is most widely used today and why.
- **[15]** Discuss network topologies in detail with diagrams. Compare them on cost, reliability, fault isolation, scalability and performance, and recommend a topology for (a) a 30-seat computer laboratory and (b) a bank's inter-branch backbone, justifying each.

## MCQ traps

| Trap                                    | Correct answer                                     |
|-----------------------------------------|----------------------------------------------------|
| "Mesh links for n nodes?"               | $n(n-1)/2$ .                                       |
| "Which topology needs terminators?"     | <b>Bus</b> .                                       |
| "Which has a central point of failure?" | <b>Star</b> (hub/switch).                          |
| "Which is most reliable?"               | <b>Mesh</b> .                                      |
| "Which is most common in modern LANs?"  | <b>Star</b> (with a switch).                       |
| "Token Ring standard?"                  | <b>IEEE 802.5</b> ; FDDI uses a <b>dual ring</b> . |
| "Ethernet on a hub is logically a?"     | <b>Bus</b> , though physically a star.             |
| "Adding a node is easiest in?"          | <b>Star</b> . Hardest in <b>ring/mesh</b> .        |

## 5. Network architectures — peer-to-peer vs client-server

### Concept

This is about the **relationship between the machines**, not the wiring. In a **peer-to-peer** network every computer is an equal — each can act as both a client (requesting resources) and a server (providing them), and each user administers their own machine. In a **client-server** network, roles are specialised: dedicated **servers** hold the resources and run the services, and **clients** only request. Administration is **centralised**.

| Criterion       | Peer-to-peer (P2P)                                         | Client-server                                        |
|-----------------|------------------------------------------------------------|------------------------------------------------------|
| Roles           | Every node is <b>both</b> client and server                | <b>Dedicated</b> servers; clients only request       |
| Number of nodes | Best for <b>small networks</b> (< ~10–15)                  | Scales to <b>thousands</b>                           |
| Cost            | <b>Low</b> — no dedicated server hardware or NOS licences  | <b>High</b> — server hardware, software, admin staff |
| Administration  | <b>Decentralised</b> — each user manages their own machine | <b>Centralised</b> — one point of control            |

|                  |                                                                              |                                                                                  |
|------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Security         | <b>Weak</b> — each machine sets its own shares/passwords                     | <b>Strong</b> — central authentication (directory service), central policy       |
| Backup           | Each user's responsibility → unreliable                                      | Centralised, systematic                                                          |
| Performance      | Degrades as nodes and sharing increase                                       | Consistent; can be tuned/updated at the server                                   |
| Reliability      | No single point of failure; but a resource is unavailable if its host is off | <b>Server failure disrupts everyone</b> (mitigate with clustering/redundancy)    |
| Setup complexity | Very easy                                                                    | Requires expertise                                                               |
| Examples         | Windows workgroup, home file/printer sharing, BitTorrent, blockchain         | Web (browser–web server), email, DNS, databases, Windows Active Directory domain |

**Server types** to name in an answer: file server, print server, database server, web server, mail server, application server, proxy server, DNS server, DHCP server, authentication server.

**A three-tier client-server architecture** (worth mentioning for 10 marks): presentation tier (client/browser) → application/business-logic tier (application server) → data tier (database server). It separates concerns, improves scalability and lets each tier be secured and scaled independently, versus the **two-tier** model where the client talks directly to the database.

## Likely exam questions

- **[5]** Differentiate between peer-to-peer and client-server network architectures.
- **[5]** What is a server? Name any five types of server.
- **[10]** Explain peer-to-peer and client-server architectures with diagrams. Compare them on cost, security, scalability and administration, and state where each is appropriate.

## MCQ traps

| Trap                                  | Correct answer                   |
|---------------------------------------|----------------------------------|
| "Which offers centralised security?"  | <b>Client-server.</b>            |
| "Which is cheaper for a 5-PC office?" | <b>Peer-to-peer.</b>             |
| "In P2P, can a node be a server?"     | <b>Yes — every node is both.</b> |
| "BitTorrent is?"                      | <b>P2P.</b>                      |

## 6. Connectivity devices — mapped to OSI layers

### Concept

Every internetworking device does exactly one thing: it **reads a header and makes a decision**. The *layer* a device operates at is simply **the highest header it reads**. A repeater reads nothing (it just amplifies) → Layer 1. A switch reads the MAC header → Layer 2. A router reads the IP header → Layer 3. Once you internalise that rule, the whole table becomes derivable instead of memorised.

### The master device table

| Device                    | OSI layer                             | Reads / forwards on     | What it does                                                                                                                                                                                                               | Collision domains                        | Broadcast domains                                 |
|---------------------------|---------------------------------------|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|---------------------------------------------------|
| Repeater                  | 1 — Physical                          | Nothing (bit stream)    | <b>Regenerates and retimes</b> a weakened digital signal to extend distance. Not merely an amplifier — an amplifier would boost the noise too; a repeater <b>reconstructs</b> the bits                                     | 1 (one shared domain)                    | 1                                                 |
| Hub (multiport repeater)  | 1 — Physical                          | Nothing                 | Receives a frame on one port and <b>floods it out of every other port</b> . No filtering, no addresses. Half-duplex, <b>CSMA/CD</b> . Obsolete                                                                             | 1 — all ports share one collision domain | 1                                                 |
| Bridge                    | 2 — Data Link                         | MAC address             | Connects <b>two LAN segments</b> and <b>filters</b> : learns which MACs are on which side and forwards a frame only if the destination is on the other side. Software-based, few ports                                     | One per port                             | 1                                                 |
| Switch (multiport bridge) | 2 — Data Link (L3 switches also do 3) | MAC address             | Builds a <b>MAC address table (CAM table)</b> by learning source addresses; forwards each frame <b>only to the destination port</b> (unicast), floods only unknown/broadcast/multicast. Hardware (ASIC) based, full duplex | One per port — this is the key advantage | 1 (one per VLAN)                                  |
| Router                    | 3 — Network                           | IP address              | Connects <b>different networks</b> ; makes forwarding decisions from a <b>routing table</b> ; determines the <b>best path</b> ; performs <b>fragmentation</b> , TTL decrement; <b>blocks broadcasts</b>                    | One per port                             | One per port — routers break up broadcast domains |
| Gateway                   | All 7 (up to Application)             | Everything — translates | A <b>protocol converter</b> : joins two networks using <b>completely different protocol architectures</b> (e.g. TCP/IP ↔ SNA, or an email gateway between SMTP and X.400). Slowest device; usually software on a server    | —                                        | —                                                 |
| Modem                     | 1 — Physical                          | —                       | <b>Mo</b> dulator– <b>dem</b> odulator: converts <b>digital</b> → <b>analogue</b> for transmission over an analogue line and back. Enables digital data over the telephone network                                         | —                                        | —                                                 |
| NIC (network adapter)     | 1 and 2                               | —                       | Physical interface; holds the <b>burned-in MAC address</b> ; frames/deframes                                                                                                                                               | —                                        | —                                                 |

|                          |                        |                        |                                                                                          |                  |   |
|--------------------------|------------------------|------------------------|------------------------------------------------------------------------------------------|------------------|---|
| <b>Access Point (AP)</b> | 2                      | MAC                    | Bridges a wireless (802.11) segment to a wired LAN; uses <b>CSMA/CA</b>                  | 1 (shared radio) | 1 |
| <b>Router</b>            | 2 and 3                | Both                   | Routes routable protocols, bridges non-routable ones                                     | —                | — |
| <b>Proxy server</b>      | <b>7 — Application</b> | Application data       | Sits between clients and the Internet: caches, filters content, hides internal addresses | —                | — |
| <b>Firewall</b>          | 3–7 (depends on type)  | Headers and/or payload | Filters traffic against a rule set                                                       | —                | — |

## The two sentences that earn marks

- **"A switch breaks up collision domains; a router breaks up both collision domains and broadcast domains."** This one sentence answers a whole family of questions. Corollary: a hub breaks up neither.
- **"A hub floods, a switch forwards, a router routes."**

*Worked example:* A 24-port switch has **24 collision domains** and **1 broadcast domain**. A 24-port hub has **1** collision domain and **1** broadcast domain. A router with 4 interfaces gives **4** broadcast domains. If three 8-port hubs are each plugged into one port of a switch, you have 3 collision domains (one per hub) plus the remaining switch ports, and still one broadcast domain.

### Hub vs switch vs router — the standard 5-mark table:

|                   | Hub                           | Switch                             | Router                       |
|-------------------|-------------------------------|------------------------------------|------------------------------|
| OSI layer         | 1                             | 2                                  | 3                            |
| Address used      | None                          | MAC                                | IP                           |
| Forwarding        | <b>Broadcast to all ports</b> | <b>Unicast to the correct port</b> | Best path from routing table |
| Table maintained  | None                          | MAC/CAM table                      | Routing table                |
| Duplex            | Half                          | Full                               | Full                         |
| Collision domains | 1                             | Per port                           | Per port                     |
| Broadcast domains | 1                             | 1 (per VLAN)                       | <b>Per port</b>              |
| Connects          | Devices in one LAN            | Devices in one LAN                 | <b>Different networks</b>    |
| Speed/cost        | Cheapest, slowest             | Fast, moderate                     | Slower per packet, costliest |

### Switching methods inside a switch (a nice extra for 10 marks):

| Method                   | Behaviour                                                         | Latency | Error checking                 |
|--------------------------|-------------------------------------------------------------------|---------|--------------------------------|
| <b>Store-and-forward</b> | Receives the whole frame, verifies the <b>CRC</b> , then forwards | Highest | <b>Full</b> — drops bad frames |

|                      |                                                                      |               |                                    |
|----------------------|----------------------------------------------------------------------|---------------|------------------------------------|
| <b>Cut-through</b>   | Reads only the first 6 bytes (destination MAC) and starts forwarding | <b>Lowest</b> | None — forwards corrupt frames too |
| <b>Fragment-free</b> | Reads the first 64 bytes (where most collisions show) then forwards  | Medium        | Partial                            |

## Likely exam questions

- **[5]** Differentiate between a hub and a switch.
- **[5]** Explain the function of a router. At which OSI layer does it operate?
- **[5]** What is a gateway? How does it differ from a router?
- **[5]** What is a modem? Explain modulation and demodulation.
- **[5]** Define collision domain and broadcast domain. How many of each does a 16-port switch create?
- **[10]** Explain the various network connectivity devices — repeater, hub, bridge, switch, router, gateway and modem — mapping each to its OSI layer and stating its function.
- **[15]** Discuss internetworking devices in detail. Explain how each operates, map them to the OSI model, explain collision and broadcast domains with examples, and compare hub, switch and router in a table.

## MCQ traps

| Trap                                      | Correct answer                                                      |
|-------------------------------------------|---------------------------------------------------------------------|
| "A switch operates at layer?"             | <b>2</b> (Data Link). Router = <b>3</b> . Hub/repeater = <b>1</b> . |
| "Which device breaks broadcast domains?"  | <b>Router</b> (not switch).                                         |
| "Which device works at all seven layers?" | <b>Gateway</b> .                                                    |
| "A bridge uses which address?"            | <b>MAC</b> .                                                        |
| "Multiport repeater = ?"                  | <b>Hub</b> . Multiport bridge = <b>switch</b> .                     |
| "Which device does fragmentation?"        | <b>Router</b> .                                                     |
| "A hub is full-duplex?"                   | <b>No — half-duplex</b> , with collisions.                          |
| "Modem converts?"                         | Digital ↔ analogue.                                                 |

## 7. The OSI reference model — in full

### Concept

The problem OSI solves is this: building a network in one piece is impossible, because a change to the cabling would force a change to your email program. So the ISO (International Organization for Standardization) published, in **1984**, the **Open Systems Interconnection** model: a **seven-layer** architecture in which each layer performs a well-defined function, uses only the services of the layer immediately below, and provides services only to the layer immediately above. Replace the physical medium and only Layer 1 changes.

Two vocabulary items to state early:

- **Peer-to-peer communication:** layer *n* on the sender logically talks to layer *n* on the

receiver, using a **protocol**. Physically, of course, the data goes all the way down, across, and back up.

- **Encapsulation**: each layer adds its own **header** (and Layer 2 adds a **trailer**) to the unit received from above. That growing unit is the layer's **PDU** (Protocol Data Unit). The receiver strips the headers in reverse — **de-encapsulation**.

**Mnemonics** — learn one in each direction:

- Layer 7 → 1: **All People Seem To Need Data Processing**
- Layer 1 → 7: **Please Do Not Throw Sausage Pizza Away**

## Diagram to hand-draw — the OSI stack with encapsulation

Draw **two vertical stacks of seven boxes** (sender on the left, receiver on the right), numbered 7 at the top down to 1 at the bottom. Join the two Layer-1 boxes with a horizontal line labelled "**physical medium**". Draw **dashed horizontal arrows** between each matching pair of layers labelled "peer protocol". Down the left side, show the data unit growing:

|           |  |                                     |  |                                       |
|-----------|--|-------------------------------------|--|---------------------------------------|
| Layer 7-5 |  | DATA                                |  | ← PDU: Data                           |
| Layer 4   |  | [TCP hdr] DATA                      |  | ← PDU: Segment (TCP) / Datagram (UDP) |
| Layer 3   |  | [IP hdr][TCP hdr] DATA              |  | ← PDU: Packet                         |
| Layer 2   |  | [Frame hdr][IP][TCP] DATA [Trailer] |  | ← PDU: Frame                          |
| Layer 1   |  | 1010110010110101110101...           |  | ← PDU: Bits                           |

That encapsulation strip is often worth 4–5 marks on its own. Draw it every time.

## The seven layers in detail

### Layer 1 — Physical

**Job**: transmit **raw bits** over a physical medium. It has no idea what the bits mean.

| Aspect              | Detail                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PDU                 | Bits                                                                                                                                                                                                                                                                                                                                                                         |
| Functions           | Bit representation (encoding: NRZ, Manchester); <b>data rate</b> (bit duration); <b>bit synchronisation</b> (sender and receiver clocks); <b>transmission mode</b> (simplex/half/full duplex); <b>physical topology</b> ; line configuration (point-to-point vs multipoint); mechanical and electrical specifications (voltage levels, pin layout, cable and connector type) |
| Devices             | Repeater, <b>hub</b> , cables, connectors, NIC (physical part), <b>modem</b> , transceiver                                                                                                                                                                                                                                                                                   |
| Standards/protocols | RS-232, V.35, X.21, RJ-45, DSL, SONET/SDH, IEEE 802.3 physical, Bluetooth PHY                                                                                                                                                                                                                                                                                                |
| Address             | None                                                                                                                                                                                                                                                                                                                                                                         |

### Layer 2 — Data Link

**Job**: turn the unreliable raw bit pipe into a **reliable link between two directly connected nodes** — hop-to-hop, node-to-node delivery.

| Aspect | Detail |
|--------|--------|
| PDU    | Frame  |

|                   |                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Functions</b>  | <b>Framing</b> (marking where a frame begins and ends); <b>physical (MAC) addressing</b> ; <b>flow control</b> (stop-and-wait, sliding window — preventing a fast sender from swamping a slow receiver); <b>error control</b> — detection via <b>CRC/checksum/parity</b> and correction via <b>retransmission (ARQ)</b> ; <b>access control</b> — deciding who may transmit on a shared medium |
| <b>Sub-layers</b> | <b>LLC</b> (Logical Link Control, IEEE <b>802.2</b> ) — interfaces to Layer 3, flow/error control. <b>MAC</b> (Media Access Control) — physical addressing and channel access (CSMA/CD, CSMA/CA, token passing)                                                                                                                                                                                |
| <b>Devices</b>    | <b>Switch</b> , bridge, NIC, wireless access point                                                                                                                                                                                                                                                                                                                                             |
| <b>Protocols</b>  | Ethernet (802.3), Wi-Fi (802.11), PPP, HDLC, SLIP, Frame Relay, ATM, Token Ring (802.5), <b>ARP</b> (sits between 2 and 3)                                                                                                                                                                                                                                                                     |
| <b>Address</b>    | <b>MAC address</b> (48-bit physical address)                                                                                                                                                                                                                                                                                                                                                   |

### Layer 3 — Network

**Job:** deliver a packet **end to end, across multiple networks** — source host to destination host, possibly through many intermediate routers. Layer 2 gets you across one hop; Layer 3 gets you across the internetwork.

| Aspect           | Detail                                                                                                                                                                                                                                                                                                     |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PDU</b>       | <b>Packet</b> (or datagram)                                                                                                                                                                                                                                                                                |
| <b>Functions</b> | <b>Logical addressing</b> (IP); <b>routing</b> — choosing the path (routing tables, routing algorithms); <b>forwarding</b> ; <b>fragmentation and reassembly</b> (when the next link's MTU is too small); <b>congestion control</b> ; internetworking; TTL/hop-count management; <b>Quality of Service</b> |
| <b>Devices</b>   | <b>Router</b> , Layer-3 switch                                                                                                                                                                                                                                                                             |
| <b>Protocols</b> | <b>IP</b> (IPv4/IPv6), <b>ICMP</b> (error reporting — <b>ping</b> , <b>traceroute</b> ), IGMP, <b>ARP/RARP</b> , routing protocols <b>RIP</b> , <b>OSPF</b> , <b>EIGRP</b> , <b>BGP</b> , IPsec                                                                                                            |
| <b>Address</b>   | <b>IP address</b> (logical)                                                                                                                                                                                                                                                                                |

### Layer 4 — Transport

**Job:** deliver data reliably (or not) **process to process**. Layer 3 gets a packet to the right *host*; Layer 4 gets it to the right *program on that host*, using **port numbers**. This is the boundary layer — **the first truly end-to-end layer**, and the lowest layer that intermediate routers do not look at.

| Aspect           | Detail                                                                                                                                                                                                                                                                                                                                                                |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PDU</b>       | <b>Segment</b> (TCP) / <b>Datagram</b> (UDP)                                                                                                                                                                                                                                                                                                                          |
| <b>Functions</b> | <b>Service-point (port) addressing</b> ; <b>segmentation and reassembly</b> (with sequence numbers); <b>connection control</b> (connection-oriented vs connectionless); <b>end-to-end flow control</b> (sliding window); <b>end-to-end error control</b> (checksum, acknowledgement, retransmission); multiplexing/demultiplexing several conversations onto one host |
| <b>Devices</b>   | Gateway, some firewalls, load balancers                                                                                                                                                                                                                                                                                                                               |
| <b>Protocols</b> | <b>TCP</b> , <b>UDP</b> , SCTP, DCCP                                                                                                                                                                                                                                                                                                                                  |
| <b>Address</b>   | <b>Port number</b> (16-bit)                                                                                                                                                                                                                                                                                                                                           |

## Layer 5 — Session

**Job:** establish, manage and terminate the **dialogue** between two applications.

| Aspect    | Detail                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PDU       | Data                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Functions | <b>Session establishment, maintenance and termination</b> ; <b>dialogue control</b> — deciding whose turn it is (half-duplex or full-duplex conversation); <b>synchronisation</b> — inserting <b>checkpoints</b> into a long transfer so that after a crash you resume from the last checkpoint rather than restarting (the classic example: a 2000-page file transfer with a checkpoint every 100 pages); token management |
| Protocols | NetBIOS, RPC, PPTP, SQL sessions, NFS, SAP, session part of SIP                                                                                                                                                                                                                                                                                                                                                             |

## Layer 6 — Presentation

**Job:** deal with the **syntax and semantics** of the data — make sure what the sender means is what the receiver understands. Sometimes called the **translation layer**.

| Aspect            | Detail                                                                                                                                                                                                                                                           |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PDU               | Data                                                                                                                                                                                                                                                             |
| Functions         | <b>Translation</b> between different data representations (ASCII ↔ EBCDIC, big-endian ↔ little-endian, into a common transfer syntax); <b>encryption and decryption</b> for confidentiality; <b>compression and decompression</b> to reduce the bits transmitted |
| Protocols/formats | <b>SSL/TLS</b> (commonly placed here), MIME, JPEG, GIF, MPEG, ASCII, EBCDIC, XDR, ASN.1                                                                                                                                                                          |

**Encryption and compression are the Presentation layer's job** — the single most examined fact about Layer 6.

## Layer 7 — Application

**Job:** provide the **interface through which user applications access network services**.

Note carefully: **the layer is not the application itself**. Chrome is not Layer 7; the **HTTP protocol** Chrome speaks is.

| Aspect    | Detail                                                                                                                                                                                                   |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PDU       | Data / message                                                                                                                                                                                           |
| Functions | Network virtual terminal (Telnet); <b>file transfer, access and management</b> (FTP); <b>mail services</b> (SMTP); <b>directory services</b> (DNS, LDAP); web browsing (HTTP); network management (SNMP) |
| Protocols | <b>HTTP/HTTPS, FTP, TFTP, SMTP, POP3, IMAP, DNS, DHCP, SNMP, Telnet, SSH, NFS, LDAP</b>                                                                                                                  |

## The consolidated OSI table — reproduce this from memory

| # | Layer       | PDU  | Address | Key functions                      | Devices        | Protocols                                     |
|---|-------------|------|---------|------------------------------------|----------------|-----------------------------------------------|
| 7 | Application | Data | —       | User interface to network services | Gateway, proxy | HTTP, FTP, SMTP, DNS, DHCP, Telnet, SNMP, SSH |

|   |                     |                          |                    |                                                                                        |                                    |                                            |
|---|---------------------|--------------------------|--------------------|----------------------------------------------------------------------------------------|------------------------------------|--------------------------------------------|
| 6 | <b>Presentation</b> | Data                     | —                  | <b>Translation, encryption, compression</b>                                            | Gateway                            | SSL/TLS, MIME, JPEG, MPEG, ASCII           |
| 5 | <b>Session</b>      | Data                     | —                  | Session setup/manage/teardown, <b>dialogue control, synchronisation (checkpoints)</b>  | Gateway                            | NetBIOS, RPC, PTP, SQL                     |
| 4 | <b>Transport</b>    | <b>Segment/ Datagram</b> | <b>Port</b>        | <b>End-to-end</b> delivery, segmentation, flow & error control, connection control     | Firewall, gateway                  | <b>TCP, UDP, SCTP</b>                      |
| 3 | <b>Network</b>      | <b>Packet</b>            | <b>IP address</b>  | <b>Logical addressing, routing, fragmentation,</b> congestion control                  | <b>Router</b>                      | <b>IP, ICMP, IGMP, ARP, RIP, OSPF, BGP</b> |
| 2 | <b>Data Link</b>    | <b>Frame</b>             | <b>MAC address</b> | <b>Framing, physical addressing, error detection (CRC), flow control, media access</b> | <b>Switch, bridge, NIC</b>         | Ethernet, PPP, HDLC, 802.11, Frame Relay   |
| 1 | <b>Physical</b>     | <b>Bits</b>              | —                  | Bit transmission, encoding, data rate, synchronisation, topology, media specs          | <b>Hub, repeater,</b> cable, modem | RS-232, V.35, DSL, SONET, 802.3 PHY        |

**Layer grouping:** Layers 1–3 (sometimes 1–4) are the **network support / media layers** — they move data. Layers 5–7 are the **user support / host layers** — they make it meaningful. Layer 4 is the **bridge** between the two.

## Merits and demerits of OSI

| Advantages                                                                         | Disadvantages                                                                                |
|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>Standardised</b> — vendor-independent interoperability                          | <b>Never fully implemented</b> — TCP/IP won in practice                                      |
| <b>Modular</b> — a change in one layer doesn't force changes elsewhere             | Layers 5 and 6 are <b>thin/underused</b> , layers 2 and 3 overloaded                         |
| <b>Simplifies teaching, design and troubleshooting</b> (isolate faults to a layer) | The model was <b>defined before the protocols</b> , so it fits nothing exactly               |
| Supports both connection-oriented and connectionless service at the network layer  | <b>Complex</b> ; overhead of headers at every layer                                          |
| Clear division of labour; encourages competition at each layer                     | Some functions (addressing, flow control, error control) are <b>duplicated</b> across layers |

## Likely exam questions

- [5] What is the OSI model? Name the seven layers in order.
- [5] Explain the functions of the Data Link layer.
- [5] Explain the functions of the Transport layer. Name its PDU and its protocols.
- [5] What is encapsulation? Explain with the PDU at each layer.

- **[5]** At which layer does encryption take place? Justify.
- **[10]** Explain the OSI reference model with a neat diagram, describing the function of each of the seven layers.
- **[15]** Describe the OSI reference model in full detail. For each layer, state its function, its PDU, the addresses it uses, the devices that operate there and its representative protocols. Explain encapsulation with a diagram, and discuss the merits and demerits of the model.

## MCQ traps

| Trap                                                 | Correct answer                                                                                  |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| "How many layers in OSI? Who developed it?"          | <b>7, ISO, in 1984.</b>                                                                         |
| "PDU at the Network layer?"                          | <b>Packet.</b> Transport = <b>segment</b> , Data Link = <b>frame</b> , Physical = <b>bits</b> . |
| "Which layer does routing?"                          | <b>Network (3).</b>                                                                             |
| "Which layer does error <i>detection</i> using CRC?" | <b>Data Link (2).</b> End-to-end error control → Transport (4).                                 |
| "Encryption/compression layer?"                      | <b>Presentation (6).</b>                                                                        |
| "Dialogue control and checkpoints?"                  | <b>Session (5).</b>                                                                             |
| "Which is the first end-to-end layer?"               | <b>Transport (4).</b>                                                                           |
| "Which layer does flow control?"                     | <b>Both 2 (hop) and 4 (end-to-end)</b> — read the question carefully.                           |
| "Which layer has no header?"                         | <b>Physical</b> (it transmits bits).                                                            |
| "Fragmentation is done at?"                          | <b>Network layer</b> , by the router.                                                           |
| "Is the web browser layer 7?"                        | The <b>protocol (HTTP)</b> is; the application program sits <i>above</i> the model.             |

## 8. The TCP/IP model

### Concept

TCP/IP came first (ARPANET, 1970s) and OSI came later as a tidy abstraction. That is why TCP/IP is untidy but real: it has **four layers** (some texts say five, splitting Network Access into Physical and Data Link), and its layers were **derived from working protocols** rather than designed in advance.

| TCP/IP layer                    | Maps to OSI | Function                                                      | Protocols                                                       |
|---------------------------------|-------------|---------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Application</b>              | 7 + 6 + 5   | All host-level functions: user services, translation, session | HTTP, FTP, SMTP, POP3, IMAP, DNS, DHCP, Telnet, SSH, SNMP, TFTP |
| <b>Transport</b> (Host-to-Host) | 4           | End-to-end delivery, ports, reliability                       | <b>TCP, UDP</b>                                                 |
| <b>Internet</b>                 | 3           | Logical addressing and routing                                | <b>IP</b> , ICMP, IGMP, ARP, RARP                               |

|                                                  |       |                                                |                                                    |
|--------------------------------------------------|-------|------------------------------------------------|----------------------------------------------------|
| <b>Network Access</b> (Link / Network Interface) | 2 + 1 | Framing, MAC addressing, physical transmission | Ethernet, Wi-Fi, PPP, Frame Relay, ATM, token ring |
|--------------------------------------------------|-------|------------------------------------------------|----------------------------------------------------|

## OSI vs TCP/IP — the comparison table

| Criterion                | OSI                                                | TCP/IP                                                             |
|--------------------------|----------------------------------------------------|--------------------------------------------------------------------|
| Number of layers         | <b>7</b>                                           | <b>4</b> (or 5)                                                    |
| Developed by             | <b>ISO</b>                                         | <b>DARPA / DoD (US Department of Defense)</b>                      |
| Order of creation        | <b>Model first, protocols after</b>                | <b>Protocols first, model after</b>                                |
| Nature                   | A <b>reference/theoretical</b> model               | An <b>implemented, practical</b> protocol suite                    |
| Layer independence       | Strictly independent; well-defined interfaces      | Layers less strictly separated                                     |
| Network layer service    | <b>Both connection-oriented and connectionless</b> | <b>Connectionless only</b> (IP)                                    |
| Transport layer service  | <b>Connection-oriented only</b>                    | <b>Both</b> — TCP (connection-oriented) and UDP (connectionless)   |
| Session and Presentation | <b>Separate layers (5, 6)</b>                      | <b>Absent</b> — merged into Application                            |
| Protocol dependence      | <b>Protocol-independent</b> — a generic standard   | <b>Protocol-dependent</b> — built around TCP and IP                |
| Usage today              | Teaching, reference, troubleshooting vocabulary    | <b>The actual Internet</b>                                         |
| Reliability of the base  | Guaranteed at multiple layers                      | Reliability is the <b>Transport layer's</b> job; IP is best-effort |

## Common port numbers — pure MCQ material

| Port           | Protocol                           | Transport                                                             |
|----------------|------------------------------------|-----------------------------------------------------------------------|
| <b>20 / 21</b> | <b>FTP</b> (20 data, 21 control)   | TCP                                                                   |
| <b>22</b>      | <b>SSH / SFTP / SCP</b>            | TCP                                                                   |
| <b>23</b>      | <b>Telnet</b>                      | TCP                                                                   |
| <b>25</b>      | <b>SMTP</b>                        | TCP                                                                   |
| <b>53</b>      | <b>DNS</b>                         | <b>UDP</b> (queries) and <b>TCP</b> (zone transfers, large responses) |
| <b>67 / 68</b> | <b>DHCP</b> (67 server, 68 client) | UDP                                                                   |
| <b>69</b>      | <b>TFTP</b>                        | UDP                                                                   |
| <b>80</b>      | <b>HTTP</b>                        | TCP                                                                   |

|                    |                             |     |
|--------------------|-----------------------------|-----|
| 110                | POP3                        | TCP |
| 119                | NNTP                        | TCP |
| 143                | IMAP                        | TCP |
| 161 / 162          | SNMP / SNMP trap            | UDP |
| 443                | HTTPS                       | TCP |
| 445                | SMB                         | TCP |
| 465 / 587          | SMTPS / SMTP submission     | TCP |
| 993 / 995          | IMAPS / POP3S               | TCP |
| 3306 / 1521 / 5432 | MySQL / Oracle / PostgreSQL | TCP |
| 3389               | RDP                         | TCP |

Port ranges: **0–1023 well-known, 1024–49151 registered, 49152–65535 dynamic/private (ephemeral).**

## MCQ traps

| Trap                                        | Correct answer                                                |
|---------------------------------------------|---------------------------------------------------------------|
| "How many layers in TCP/IP?"                | <b>4</b> (Application, Transport, Internet, Network Access).  |
| "Which OSI layers are missing from TCP/IP?" | <b>Session and Presentation</b> (absorbed into Application).  |
| "Which model came first?"                   | <b>TCP/IP protocols</b> ; OSI model was standardised in 1984. |
| "DNS uses which transport?"                 | <b>UDP port 53</b> normally, TCP for zone transfers.          |
| "Port 25 / 110 / 143?"                      | SMTP / POP3 / IMAP.                                           |
| "TCP/IP internet layer is connectionless?"  | <b>Yes</b> — IP is best-effort.                               |

## 9. IP addressing — with worked subnetting

**This is the highest-scoring numerical topic in the paper. Work every example by hand.**

### Concept

An **IPv4 address** is a **32-bit logical address** identifying an interface on a network. It is written in **dotted decimal** as four **octets** (0–255): **192.168.10.25**. Every address splits into two parts:

NETWORK ID

HOST ID

The **network ID** says *which network*; the **host ID** says *which machine on it*. Routers care only about the network part — which is exactly why the split exists: it keeps routing tables small.

**How does a router know where the split is?** From the **subnet mask** — a 32-bit number with **1s over the network part and 0s over the host part**, contiguously. Written as `255.255.255.0` or as **slash/CIDR notation** `/24` (the count of 1 bits).

**The AND operation** is the whole mechanism: to find the network address of a host, do a **bitwise AND of the IP address with the subnet mask**. Say this in every subnetting answer.

## Address classes (classful addressing)

| Class | Leading bits | First octet range | Default mask        | Network/Host split | Networks             | Hosts per network         | Purpose                            |
|-------|--------------|-------------------|---------------------|--------------------|----------------------|---------------------------|------------------------------------|
| A     | 0            | 1 – 126           | 255.0.0.0 (/8)      | N.H.H.H            | $2^7 - 2 = 126$      | $2^{24} - 2 = 16,777,214$ | Very large organisations           |
| B     | 10           | 128 – 191         | 255.255.0.0 (/16)   | N.N.H.H            | $2^{14} = 16,384$    | $2^{16} - 2 = 65,534$     | Medium/large                       |
| C     | 110          | 192 – 223         | 255.255.255.0 (/24) | N.N.N.H            | $2^{21} = 2,097,152$ | $2^8 - 2 = 254$           | Small networks                     |
| D     | 1110         | 224 – 239         | —                   | —                  | —                    | —                         | <b>Multicast</b> (no host portion) |
| E     | 1111         | 240 – 255         | —                   | —                  | —                    | —                         | <b>Experimental / reserved</b>     |

**Why 127 is missing from Class A:** `127.0.0.0/8` is reserved for **loopback** (`127.0.0.1` = localhost). And why "-2" in the host counts: **the all-zeros host address is the network address** and **the all-ones host address is the broadcast address**; neither can be assigned to a machine.

## Special and reserved addresses

| Address                                                | Meaning                                                                                       |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>0.0.0.0</code>                                   | "This host" / unspecified; also the <b>default route</b> in a routing table                   |
| <code>127.0.0.1</code><br>( <code>127.0.0.0/8</code> ) | <b>Loopback</b> — traffic never leaves the machine                                            |
| <code>255.255.255.255</code>                           | <b>Limited broadcast</b> — this network only, never forwarded by routers                      |
| Network ID (host bits all 0)                           | Identifies the network itself; <b>not assignable</b>                                          |
| Broadcast (host bits all 1)                            | <b>Directed broadcast</b> to all hosts on that network; <b>not assignable</b>                 |
| <code>169.254.0.0/16</code>                            | <b>APIPA / link-local</b> — self-assigned when DHCP fails. Seeing this means "no DHCP server" |
| <code>224.0.0.0/4</code>                               | Multicast                                                                                     |

## Private address ranges — memorise exactly (RFC 1918)

| Class | Range                         | CIDR           | Number of addresses |
|-------|-------------------------------|----------------|---------------------|
| A     | 10.0.0.0 – 10.255.255.255     | 10.0.0.0/8     | 16,777,216          |
| B     | 172.16.0.0 – 172.31.255.255   | 172.16.0.0/12  | 1,048,576           |
| C     | 192.168.0.0 – 192.168.255.255 | 192.168.0.0/16 | 65,536              |

The Class B range is the one people get wrong: it is **172.16 through 172.31**, *not* 172.16 through 172.255 and *not* only 172.16. Private addresses are **not routable on the Internet**; **NAT** (Network Address Translation) on the edge router translates them to one or a few public addresses, which is why the world has not yet run out of IPv4. **PAT / NAT overload** adds port numbers so that thousands of internal hosts can share **one** public address.

## Subnetting

**Why subnet?** A single Class B network with 65,534 hosts is unusable: one broadcast reaches all 65,534 machines, security is impossible, and troubleshooting is hopeless. **Subnetting borrows bits from the host portion and gives them to the network portion**, creating multiple smaller networks inside one address block. Benefits: reduced broadcast traffic, better security/isolation, easier management, efficient address use.

**The four formulas — write them at the top of your answer sheet:**

| Quantity                             | Formula                                                                                             |
|--------------------------------------|-----------------------------------------------------------------------------------------------------|
| Number of subnets                    | $2^n$ , where <b>n = number of borrowed (subnet) bits</b>                                           |
| Number of usable hosts per subnet    | $2^h - 2$ , where <b>h = number of remaining host bits</b>                                          |
| <b>Block size</b> (subnet increment) | <b>256 – (value of the interesting octet in the mask)</b> , or equivalently $2^h$ within that octet |
| Valid host range                     | (network + 1) to (broadcast – 1)                                                                    |

The "**interesting octet**" is the last octet of the mask that is neither 255 nor 0.

**Mask cheat sheet — learn this cold; it makes every question a lookup:**

| CIDR | Mask                    | Block size | Hosts/subnet (usable)                      |
|------|-------------------------|------------|--------------------------------------------|
| /24  | 255.255.255. <b>0</b>   | 256        | 254                                        |
| /25  | 255.255.255. <b>128</b> | 128        | 126                                        |
| /26  | 255.255.255. <b>192</b> | 64         | 62                                         |
| /27  | 255.255.255. <b>224</b> | 32         | 30                                         |
| /28  | 255.255.255. <b>240</b> | 16         | 14                                         |
| /29  | 255.255.255. <b>248</b> | 8          | 6                                          |
| /30  | 255.255.255. <b>252</b> | 4          | <b>2</b> (used for router-to-router links) |

|     |                 |   |                                      |
|-----|-----------------|---|--------------------------------------|
| /31 | 255.255.255.254 | 2 | 0 (special, point-to-point RFC 3021) |
| /32 | 255.255.255.255 | 1 | single host                          |

Octet values in order: **128, 192, 224, 240, 248, 252, 254, 255**. These correspond to 1,2,3,4,5,6,7,8 bits borrowed within an octet.

## Worked subnetting example 1 — Class C, 4 subnets

**Problem:** Divide **192.168.1.0/24** into **4 subnets**. Give the mask, the subnet addresses, the broadcast addresses and the valid host ranges.

**Step 1 — bits to borrow.** We need 4 subnets, and  $2^n \geq 4 \rightarrow n = 2$ .

**Step 2 — new mask.** Default /24, borrow 2  $\rightarrow$  /26. In binary the fourth octet becomes **11000000** = 192, so the mask is **255.255.255.192**.

**Step 3 — hosts per subnet.** Remaining host bits  $h = 32 - 26 = 6$ . Usable hosts =  $2^6 - 2 = 62$ .

**Step 4 — block size.**  $256 - 192 = 64$ . So subnets increment by 64 in the last octet.

**Step 5 — tabulate.**

| Subnet | Network address      | First host    | Last host     | Broadcast            |
|--------|----------------------|---------------|---------------|----------------------|
| 1      | <b>192.168.1.0</b>   | 192.168.1.1   | 192.168.1.62  | <b>192.168.1.63</b>  |
| 2      | <b>192.168.1.64</b>  | 192.168.1.65  | 192.168.1.126 | <b>192.168.1.127</b> |
| 3      | <b>192.168.1.128</b> | 192.168.1.129 | 192.168.1.190 | <b>192.168.1.191</b> |
| 4      | <b>192.168.1.192</b> | 192.168.1.193 | 192.168.1.254 | <b>192.168.1.255</b> |

**Check:** 4 subnets  $\times$  62 usable hosts = 248 usable, plus 8 addresses lost to network/broadcast pairs = 256. ✓

## Worked subnetting example 2 — Class C, "at least 5 subnets"

**Problem:** An organisation has **192.168.10.0/24** and needs **at least 5 subnets**. Design it.

**Step 1.**  $2^n \geq 5 \rightarrow n = 2$  gives 4 (not enough),  **$n = 3$  gives 8 ✓**

**Step 2.** Mask = /24 + 3 = /27  $\rightarrow$  fourth octet **11100000** = 224  $\rightarrow$  **255.255.255.224**

**Step 3.**  $h = 32 - 27 = 5 \rightarrow$  hosts =  $2^5 - 2 = 30$  per subnet

**Step 4.** Block size =  $256 - 224 = 32$

| # | Network       | Host range | Broadcast |
|---|---------------|------------|-----------|
| 1 | 192.168.10.0  | .1 – .30   | .31       |
| 2 | 192.168.10.32 | .33 – .62  | .63       |
| 3 | 192.168.10.64 | .65 – .94  | .95       |
| 4 | 192.168.10.96 | .97 – .126 | .127      |

|   |                |             |      |
|---|----------------|-------------|------|
| 5 | 192.168.10.128 | .129 – .158 | .159 |
| 6 | 192.168.10.160 | .161 – .190 | .191 |
| 7 | 192.168.10.192 | .193 – .222 | .223 |
| 8 | 192.168.10.224 | .225 – .254 | .255 |

Five subnets are used; **three remain spare** for future growth. Note the trade-off you should comment on: 8 subnets of 30 hosts wastes fewer addresses than 4 subnets of 62 if the departments are small, but caps each subnet at 30 machines.

### Worked subnetting example 3 — find the network from a host address

**Problem:** A host has IP **192.168.20.130** with mask **255.255.255.192**. Find (a) the subnet address, (b) the broadcast address, (c) the valid host range, (d) how many hosts per subnet, (e) is **192.168.20.191** a valid host address?

**Step 1 — CIDR.** 255.255.255.192 → 192 = **11000000** = 2 bits → **/26**

**Step 2 — block size.** 256 – 192 = **64**. Subnet boundaries in the last octet:  
**0, 64, 128, 192.**

**Step 3 — locate 130.**  $128 \leq 130 < 192$  → the host lives in the **128** block.

**(a) Subnet address = 192.168.20.128**

**(b) Broadcast = next boundary – 1 = 192 – 1 = 192.168.20.191**

**(c) Valid hosts = 192.168.20.129 to 192.168.20.190**

**(d)  $2^6 - 2 = 62$  hosts**

**(e) No — .191** is the **broadcast address** of this subnet, not assignable.

**Verification by AND (show this working — it earns marks):**

|      |                 |   |                                       |
|------|-----------------|---|---------------------------------------|
| IP   | 192.168.20.130  | = | 11000000.10101000.00010100.10000010   |
| Mask | 255.255.255.192 | = | 11111111.11111111.11111111.11000000   |
| AND  | <hr/>           |   |                                       |
| Net  | 192.168.20.128  | = | 11000000.10101000.00010100.10000000 ✓ |

### Worked subnetting example 4 — Class B subnetting

**Problem:** Given **172.16.35.75/20**, find the subnet address, broadcast address, valid host range, number of subnets and hosts per subnet.

**Step 1 — the mask.** /20 = 20 ones = **11111111.11111111.11110000.00000000** = **255.255.240.0**. The **interesting octet is the third**.

**Step 2 — borrowed bits.** 172.16 is Class B (default /16), so  **$n = 20 - 16 = 4$**  borrowed →  **$2^4 = 16$  subnets**.

**Step 3 — hosts.**  $h = 32 - 20 = 12$  →  **$2^{12} - 2 = 4,094$  hosts per subnet**.

**Step 4 — block size.** 256 – 240 = **16**, applied to the **third** octet. Boundaries:  
0, 16, 32, 48, 64, 80, ..., 240.

**Step 5 — locate 35 in the third octet.**  $32 \leq 35 < 48$  → the block is **32**.

|        |       |
|--------|-------|
| Answer | Value |
|--------|-------|

| Dept             | Network                          | Mask            | First host | Last host | Broadcast   | Usable                |
|------------------|----------------------------------|-----------------|------------|-----------|-------------|-----------------------|
| <b>A</b><br>(60) | <b>192.168.1.0/26</b>            | 255.255.255.192 | .1         | .62       | <b>.63</b>  | 62                    |
| <b>B</b><br>(28) | <b>192.168.1.64/27</b>           | 255.255.255.224 | .65        | .94       | <b>.95</b>  | 30                    |
| <b>C</b><br>(12) | <b>192.168.1.96/28</b>           | 255.255.255.240 | .97        | .110      | <b>.111</b> | 14                    |
| <b>D (5)</b>     | <b>192.168.1.112/29</b>          | 255.255.255.248 | .113       | .118      | <b>.119</b> | 6                     |
| <i>Spare</i>     | 192.168.1.120 –<br>192.168.1.255 |                 |            |           |             | 136 addresses<br>free |

**Step 3 — router links.** WAN point-to-point links need exactly 2 hosts → use **/30** (255.255.255.252, block size 4). Carve them from the spare space:

192.168.1.120/30 (hosts .121, .122, broadcast .123),  
192.168.1.124/30 (hosts .125, .126, broadcast .127).

**Comment for the examiner:** classful/FLSM design at /26 would give only 4 equal subnets of 62 hosts — enough by count, but department D would waste 57 addresses and there would be no room for the /30 WAN links. **VLSM's advantage is address-space efficiency.** Its requirement: routing protocols must be **classless** (send the mask with the update) — **RIPv2, OSPF, EIGRP, BGP** support VLSM; **RIPv1 and IGRP do not.**

## Worked example 7 — supernetting / CIDR aggregation

**Problem:** Aggregate 192.168.8.0/24, 192.168.9.0/24, 192.168.10.0/24 and 192.168.11.0/24 into one route.

**Method:** convert the varying octet to binary and find the **common prefix**.

```

8 = 0000 1000
9 = 0000 1001
10 = 0000 1010
11 = 0000 1011
    ^^^^^^
    first 6 bits are common (000010)

```

Common bits in the third octet = 6. Total prefix = 8 + 8 + 6 = **/22**.

Mask = 255.255.252.0. **Supernet = 192.168.8.0/22**, covering 192.168.8.0 through 192.168.11.255 — **1,024 addresses, 1,022 usable**.

**Why this matters:** **CIDR (Classless Inter-Domain Routing, RFC 1519)** abolished the rigid A/B/C boundaries. **Route aggregation / summarisation** lets one routing-table entry replace four, which is the only reason the global BGP table is manageable. Note the requirement: **blocks must be contiguous and the starting address must be a multiple of the block size** (8 is a multiple of 4 ✓).

## IPv4 vs IPv6

IPv4's 32 bits give 4.3 billion addresses — exhausted. **IPv6** uses **128 bits**, written as eight groups of four hex digits separated by colons:  
`2001:0db8:85a3:0000:0000:8a2e:0370:7334`, shortened by dropping leading zeros in each group and replacing **one** run of all-zero groups with `::` → `2001:db8:85a3::8a2e:370:7334`. The `::` may appear **only once**, otherwise the expansion is ambiguous.

| Feature         | IPv4                                             | IPv6                                                           |
|-----------------|--------------------------------------------------|----------------------------------------------------------------|
| Address size    | <b>32 bits</b>                                   | <b>128 bits</b>                                                |
| Address space   | $2^{32} \approx 4.3 \times 10^9$                 | $2^{128} \approx 3.4 \times 10^{38}$                           |
| Notation        | Dotted <b>decimal</b> , <code>192.168.1.1</code> | Colon <b>hexadecimal</b> , <code>2001:db8::1</code>            |
| Header          | <b>Variable, 20–60 bytes</b> , 13 fields         | <b>Fixed 40 bytes</b> , 8 fields + extension headers           |
| Header checksum | <b>Present</b>                                   | <b>Removed</b> (Layer 2 and 4 already check) — faster routers  |
| Fragmentation   | By <b>sender and routers</b>                     | By the <b>source only</b> (Path MTU Discovery)                 |
| Broadcast       | <b>Yes</b>                                       | <b>No broadcast</b> — replaced by <b>multicast and anycast</b> |
| Address types   | Unicast, multicast, broadcast                    | <b>Unicast, multicast, anycast</b>                             |
| Configuration   | Manual or DHCP                                   | <b>SLAAC</b> (stateless autoconfiguration) or DHCPv6           |
| Security        | IPSec optional                                   | <b>IPSec built in</b> (mandatory in the original spec)         |
| ARP             | Uses <b>ARP</b>                                  | Uses <b>NDP</b> (Neighbor Discovery Protocol, ICMPv6)          |
| QoS             | ToS field                                        | <b>Flow Label</b> + Traffic Class                              |
| Loopback        | 127.0.0.1                                        | <code>::1</code>                                               |
| Default route   | 0.0.0.0/0                                        | <code>::/0</code>                                              |
| NAT             | Essential                                        | Largely unnecessary                                            |

Transition mechanisms to name: **dual stack**, **tunnelling** (6to4, Teredo — IPv6 packets encapsulated in IPv4), and **translation** (NAT64).

## MAC address, ARP and RARP

A **MAC address** (physical/hardware address) is a **48-bit (6-byte)** address **burned into the NIC** by the manufacturer, written as **12 hex digits**: `00:1A:2B:3C:4D:5E`. The **first 3 bytes (24 bits) are the OUI** — Organizationally Unique Identifier, assigned to the vendor; the last 3 bytes are the vendor's serial number. The broadcast MAC is **`FF:FF:FF:FF:FF:FF`**.

|       | MAC address                   | IP address                             |
|-------|-------------------------------|----------------------------------------|
| Layer | <b>2 (Data Link)</b>          | <b>3 (Network)</b>                     |
| Size  | <b>48 bits</b> (6 bytes, hex) | <b>32 bits</b> IPv4 (4 bytes, decimal) |

|             |                                                 |                                              |
|-------------|-------------------------------------------------|----------------------------------------------|
| Assigned by | <b>Manufacturer</b> , burned into hardware      | <b>Network administrator / DHCP</b>          |
| Changes?    | <b>Permanent</b> (though spoofable in software) | <b>Changes</b> when the device moves network |
| Scope       | <b>Local link only</b> — never crosses a router | <b>End to end, globally routable</b>         |
| Also called | Physical/hardware/burned-in address             | Logical address                              |

**Why both?** Here is the sentence that answers the whole question: **the IP address says where the packet is ultimately going; the MAC address says where it goes next.** As a packet crosses the internet, the **source and destination IP addresses never change**, but the **source and destination MAC addresses are rewritten at every hop** — each router strips the old frame and builds a new one for the next link.

| Protocol    | Full name                           | Maps                              | Used by                                                                           | Note                                                                                                                                                              |
|-------------|-------------------------------------|-----------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ARP</b>  | Address Resolution Protocol         | <b>IP → MAC</b>                   | A host that knows the IP of a neighbour but needs its MAC to build a frame        | Broadcasts "Who has 192.168.1.5? Tell 192.168.1.10"; the owner <b>unicasts</b> the reply. Results cached in the <b>ARP cache</b> ( <code>arp -a</code> )          |
| <b>RARP</b> | Reverse ARP                         | <b>MAC → IP</b>                   | A <b>diskless workstation</b> that knows only its own MAC and needs an IP at boot | Obsolete — replaced by <b>BOOTP</b> and then <b>DHCP</b>                                                                                                          |
| <b>ICMP</b> | Internet Control Message Protocol   | —                                 | Error reporting and diagnostics                                                   | <code>ping</code> (echo request/reply), <code>traceroute</code> (TTL exceeded), destination unreachable                                                           |
| <b>DHCP</b> | Dynamic Host Configuration Protocol | Assigns IP + mask + gateway + DNS | Automatic host configuration                                                      | Four-step <b>DORA</b> : <b>D</b> iscover (client broadcast) → <b>O</b> ffer (server) → <b>R</b> quest (client) → <b>A</b> cknowledge (server). Ports <b>67/68</b> |

**ARP spoofing / poisoning** (relevant to your Forensic paper): ARP has **no authentication**, so an attacker can send unsolicited ARP replies claiming the gateway's IP maps to the attacker's MAC. Victims then send all their traffic to the attacker — a **man-in-the-middle** attack, and the basis of most LAN packet sniffing on switched networks.

## Likely exam questions

- **[5]** What is an IP address? Explain the classes of IP addresses with their ranges.
- **[5]** What is a subnet mask? Explain the AND operation with an example.
- **[5]** List the private IP address ranges. Why are they needed?
- **[5]** Differentiate between MAC address and IP address.
- **[5]** What is ARP? How does it differ from RARP?
- **[5]** Compare IPv4 and IPv6.
- **[10]** Given the address 192.168.20.130 with mask 255.255.255.192, determine the subnet address, broadcast address, valid host range and number of hosts. Show all working.

- **[10]** Divide 192.168.1.0/24 into 8 subnets. Give the mask, and tabulate every subnet's network address, host range and broadcast address.
- **[15]** Explain IP addressing in detail: classes, subnet masks, subnetting, CIDR and VLSM. An organisation with 192.168.1.0/24 has departments needing 60, 28, 12 and 5 hosts. Design a complete VLSM addressing scheme with full working.
- **[15]** Explain classful and classless addressing. Discuss the exhaustion of IPv4 and the solutions adopted — CIDR, NAT, private addressing and IPv6 — and compare IPv4 with IPv6.

## MCQ traps

| Trap                                      | Correct answer                                                   |
|-------------------------------------------|------------------------------------------------------------------|
| "Class B first-octet range?"              | <b>128–191</b> . Class A <b>1–126</b> , Class C <b>192–223</b> . |
| "Why is 127 excluded?"                    | <b>Loopback</b> .                                                |
| "Hosts in a Class C network?"             | <b>254</b> (not 256).                                            |
| "255.255.255.240 = ?"                     | <b>/28</b> , block size 16, <b>14 hosts</b> .                    |
| "Private Class B range?"                  | <b>172.16.0.0 – 172.31.255.255</b> (not 172.16–172.255).         |
| "MAC address size?"                       | <b>48 bits</b> . IPv4 = 32, IPv6 = <b>128</b> .                  |
| "ARP maps?"                               | <b>IP → MAC</b> . RARP is the reverse.                           |
| "Number of subnets from 3 borrowed bits?" | <b>2<sup>3</sup> = 8</b> .                                       |
| "APIPA range?"                            | <b>169.254.0.0/16</b> .                                          |
| "Broadcast of 192.168.1.64/26?"           | <b>192.168.1.127</b> .                                           |
| "Mask for a 2-host WAN link?"             | <b>/30</b> — 255.255.255.252.                                    |
| "Does the IP address change at each hop?" | <b>No</b> — the <b>MAC</b> does.                                 |
| "DHCP sequence?"                          | <b>DORA</b> .                                                    |
| "IPv6 has broadcast?"                     | <b>No</b> — multicast and anycast only.                          |

## 10. Routing

### Concept

**Routing** is deciding *which path* a packet should take; **forwarding** is the act of moving it out of the right interface. A router keeps a **routing table** whose entries pair a **destination network** with a **next hop**, plus a **metric** (the cost) and often an administrative distance. When a packet arrives, the router finds the **longest matching prefix** in the table and forwards accordingly; if nothing matches, it uses the **default route** (**0.0.0.0/0**), and if there is none, it drops the packet and returns an ICMP "destination unreachable".

|                   | Static routing                                | Dynamic routing                             |
|-------------------|-----------------------------------------------|---------------------------------------------|
| Configured by     | Administrator, manually                       | Learned automatically by a routing protocol |
| Adapts to failure | <b>No</b> — needs manual change               | <b>Yes</b> — reconverges around a failure   |
| Overhead          | <b>None</b> (no protocol traffic, no CPU)     | Bandwidth and CPU consumed by updates       |
| Security          | More secure/predictable                       | Updates can be forged unless authenticated  |
| Suits             | <b>Small, stable networks</b> ; stub networks | <b>Large, changing networks</b>             |

## Distance Vector vs Link State — the central comparison

**Distance vector** ("routing by rumour"): each router knows only the **distance** (metric) and **direction** (next hop) to each destination, and it learns them by **periodically sending its entire routing table to its directly connected neighbours**. It never sees the whole topology — it simply trusts what its neighbours tell it. Algorithm: **Bellman–Ford**.

**Link state**: every router **floods** a small **Link State Advertisement** describing only **its own directly attached links to every router in the area**. Each router therefore builds an **identical complete map** of the topology (the link-state database), and then independently runs **Dijkstra's shortest-path-first algorithm** on that map to compute its own best paths.

| Criterion             | Distance Vector                                                                   | Link State                                              |
|-----------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------|
| Algorithm             | <b>Bellman–Ford</b>                                                               | <b>Dijkstra (SPF)</b>                                   |
| What is shared        | <b>The entire routing table</b>                                                   | <b>Only link states (LSAs)</b> about own links          |
| Shared with           | <b>Directly connected neighbours only</b>                                         | <b>All routers in the area</b> (flooding)               |
| When shared           | <b>Periodically</b> (RIP: every 30 s)                                             | <b>Only on change</b> (plus periodic refresh)           |
| Knowledge of topology | <b>Only what neighbours report</b> — no map                                       | <b>Complete map</b> of the area                         |
| Convergence           | <b>Slow</b>                                                                       | <b>Fast</b>                                             |
| CPU / memory          | <b>Low</b>                                                                        | <b>High</b> (must store the whole database and run SPF) |
| Bandwidth             | High periodically (whole table)                                                   | Low steady-state (small triggered updates)              |
| Loop problems         | <b>Count-to-infinity</b> , routing loops                                          | Essentially loop-free                                   |
| Loop remedies         | Split horizon, poison reverse, hold-down timers, triggered updates, max hop count | Not required                                            |
| Hierarchy             | Flat                                                                              | <b>Areas</b> (OSPF area 0 = backbone)                   |

path optimality — the reason the Internet is routable at all.

- **Broadcast, multicast and anycast routing; spanning tree** for loop-free broadcast.
- **Virtual circuits vs datagrams** — see §12.
- **Concatenated virtual circuits** (explicitly in your Forensic syllabus): when a connection crosses several different networks, a virtual circuit is set up in each network and the circuits are **spliced/concatenated end to end** at the gateways, so the whole path behaves like one connection-oriented circuit. Contrast with the **connectionless internetworking** approach (what IP actually does), where each datagram is routed independently and may take different routes and arrive out of order. Concatenated VCs preserve ordering and allow resource reservation, but fail entirely if any constituent network fails and cannot route around failures.

## Likely exam questions

- **[5]** Differentiate between static and dynamic routing.
- **[5]** Explain the count-to-infinity problem and any two solutions.
- **[5]** What is a routing table? What information does it contain?
- **[5]** Differentiate between RIP and OSPF.
- **[10]** Compare distance vector and link state routing algorithms in detail, naming the underlying algorithm, what is exchanged, with whom, how often, and the convergence and loop characteristics of each.
- **[15]** Explain routing in computer networks: routing tables, static vs dynamic, distance vector (Bellman–Ford) and link state (Dijkstra) with an illustrative topology, the count-to-infinity problem and its remedies, and the roles of RIP, OSPF and BGP.

## MCQ traps

| Trap                                        | Correct answer                                              |
|---------------------------------------------|-------------------------------------------------------------|
| "RIP maximum hop count?"                    | <b>15</b> (16 means unreachable).                           |
| "OSPF metric?"                              | <b>Cost</b> (bandwidth-based), not hop count.               |
| "Which algorithm does OSPF use?"            | <b>Dijkstra / SPF</b> . RIP uses <b>Bellman–Ford</b> .      |
| "Which is the Internet's routing protocol?" | <b>BGP</b> — a <b>path vector</b> EGP over <b>TCP 179</b> . |
| "OSPF backbone area?"                       | <b>Area 0</b> .                                             |
| "RIPv1 supports VLSM?"                      | <b>No</b> — it is classful. RIPv2 does.                     |
| "Split horizon prevents?"                   | Routing loops / count-to-infinity.                          |
| "Distance vector sends what, to whom?"      | <b>Whole table</b> , to <b>neighbours only</b> .            |

## 11. The Transport layer — TCP and UDP

## Concept

IP is **best-effort**: it will try to deliver your packet, but it makes no promise about arrival, order, duplication or corruption. Somebody has to decide whether that is acceptable. That is the Transport layer's choice, and it offers exactly two answers:

- **TCP** — "I will make it look like a reliable, ordered byte stream, whatever it costs."
- **UDP** — "I will add ports and a checksum and get out of the way."

Neither is better; they are different trade-offs of **reliability against latency and overhead**.

## TCP vs UDP — the table

| Criterion           | TCP (Transmission Control Protocol)                                                        | UDP (User Datagram Protocol)                                                                                    |
|---------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Connection          | <b>Connection-oriented</b> — handshake before data                                         | <b>Connectionless</b> — just send                                                                               |
| Reliability         | <b>Reliable</b> — acknowledgements + retransmission                                        | <b>Unreliable</b> — fire and forget                                                                             |
| Ordering            | <b>Guaranteed</b> in-order delivery (sequence numbers)                                     | <b>No ordering guarantee</b>                                                                                    |
| Duplicate detection | Yes                                                                                        | No                                                                                                              |
| Error checking      | Checksum + <b>recovery</b>                                                                 | Checksum <b>only, discards bad datagrams</b>                                                                    |
| Flow control        | <b>Yes</b> — sliding window, receiver-advertised                                           | <b>No</b>                                                                                                       |
| Congestion control  | <b>Yes</b> — slow start, congestion avoidance, fast retransmit/recovery                    | <b>No</b>                                                                                                       |
| Header size         | <b>20 bytes minimum</b> (up to 60 with options)                                            | <b>8 bytes fixed</b>                                                                                            |
| Speed / overhead    | <b>Slower, heavy</b>                                                                       | <b>Fast, lightweight</b>                                                                                        |
| Data unit (PDU)     | <b>Segment</b>                                                                             | <b>Datagram</b>                                                                                                 |
| Transmission style  | <b>Byte stream</b> (no message boundaries preserved)                                       | <b>Message/datagram oriented</b> (boundaries preserved)                                                         |
| Broadcast/multicast | <b>Not supported</b> (point-to-point only)                                                 | <b>Supported</b>                                                                                                |
| Applications        | <b>HTTP/HTTPS, FTP, SMTP, POP3, IMAP, SSH, Telnet</b> — anything where correctness matters | <b>DNS, DHCP, TFTP, SNMP, VoIP, video streaming, online gaming, RIP</b> — anything where speed beats perfection |

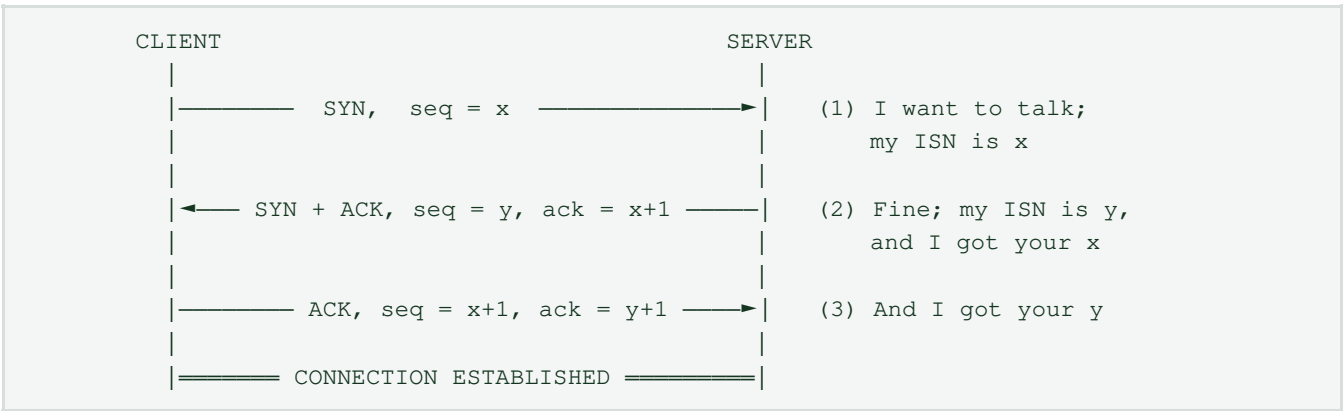
**UDP's 8-byte header** has exactly four 2-byte fields: **source port, destination port,**

**length, checksum.** Being able to name all four is an easy mark.

**TCP header key fields** (20 bytes): source port, destination port, **sequence number (32 bits)**, **acknowledgement number (32 bits)**, header length/offset, **flags (URG, ACK, PSH, RST, SYN, FIN)**, **window size**, checksum, urgent pointer, options.

**The three-way handshake — draw this**

TCP must synchronise sequence numbers in **both** directions before data flows. Two messages would establish it one way only; three is the minimum for both.



| Step | Segment         | Flags                                      | Meaning                                                           |
|------|-----------------|--------------------------------------------|-------------------------------------------------------------------|
| 1    | Client → Server | <b>SYN</b> , seq = $x$                     | Client asks to open, sends its <b>Initial Sequence Number</b> $x$ |
| 2    | Server → Client | <b>SYN + ACK</b> , seq = $y$ , ack = $x+1$ | Server agrees, sends its own ISN $y$ , and acknowledges $x$       |
| 3    | Client → Server | <b>ACK</b> , ack = $y+1$                   | Client acknowledges $y$ . Connection is now <b>ESTABLISHED</b>    |

Note the acknowledgement number is always **the next byte expected**, hence  $x+1$  not  $x$ . The SYN flag itself consumes one sequence number.

**Connection termination is a FOUR-way handshake** (FIN → ACK → FIN → ACK), because TCP connections are **full-duplex** and each direction must be closed independently — one side can finish sending while still receiving ("half-close"). Confusing three-way (setup) with four-way (teardown) is a very common MCQ trap.

**SYN flood attack** (relevant to your Forensic paper): the attacker sends thousands of SYNs with spoofed source addresses and never sends the third ACK. Each half-open connection consumes a slot in the server's backlog queue until legitimate clients cannot connect — a **denial of service**. Mitigation: **SYN cookies**, reduced timeouts, rate limiting, firewalls.

**Flow control and congestion control — distinguish them**

|  |              |                    |
|--|--------------|--------------------|
|  | Flow control | Congestion control |
|--|--------------|--------------------|

|           |                                                         |                                                                                                                                                                                                    |
|-----------|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Protects  | The <b>receiver</b> from being overwhelmed              | The <b>network</b> from being overwhelmed                                                                                                                                                          |
| Mechanism | <b>Sliding window</b> , receiver advertises <b>rwnd</b> | Sender maintains <b>cwnd</b> : <b>slow start</b> (exponential growth), <b>congestion avoidance</b> (linear, after the threshold), <b>fast retransmit</b> on 3 duplicate ACKs, <b>fast recovery</b> |
| Scope     | End to end, two parties                                 | The whole path                                                                                                                                                                                     |

Sender's usable window = **min(rwnd, cwnd)**.

**Error control mechanisms** to name: **checksum** for detection; **acknowledgement** (positive, cumulative); **retransmission on timeout** (RTO, computed from a smoothed round-trip-time estimate); ARQ schemes — **stop-and-wait**, **go-back-N**, **selective repeat**.

## Likely exam questions

- **[5]** Differentiate between TCP and UDP.
- **[5]** Explain the TCP three-way handshake with a diagram.
- **[5]** Why is TCP connection termination a four-way handshake?
- **[5]** Distinguish between flow control and congestion control.
- **[5]** Name the fields of the UDP header. Why is UDP used for DNS and VoIP?
- **[10]** Explain the functions of the Transport layer. Compare TCP and UDP in detail and explain connection establishment and termination with diagrams.
- **[15]** Explain TCP in detail: header format, three-way handshake, sliding-window flow control, congestion control (slow start and congestion avoidance) and error recovery. Contrast with UDP and state where each is appropriate.

## MCQ traps

| Trap                                    | Correct answer                                           |
|-----------------------------------------|----------------------------------------------------------|
| "TCP header minimum size?"              | <b>20 bytes</b> . UDP = <b>8 bytes</b> .                 |
| "Handshake steps for setup / teardown?" | <b>3</b> for setup, <b>4</b> for teardown.               |
| "Which flags in step 2?"                | <b>SYN + ACK</b> .                                       |
| "Ack number in step 2?"                 | <b>x + 1</b> , not x.                                    |
| "Which supports multicast?"             | <b>UDP</b> .                                             |
| "DNS uses?"                             | <b>UDP</b> (53) normally.                                |
| "Does UDP have a checksum?"             | <b>Yes</b> — but it discards bad datagrams; no recovery. |
| "TCP preserves message boundaries?"     | <b>No</b> — it is a <b>byte stream</b> . UDP does.       |

## 12. Switching

## Concept

When many nodes must be interconnected but a full mesh is impossible, we use **switching**: a network of intermediate nodes (switches) that create a path between sender and receiver on demand. There are three fundamentally different ways to do it.

## The three techniques

|                        | Circuit switching                                                                  | Message switching                                                         | Packet switching                                                   |
|------------------------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------|
| Path                   | A <b>dedicated physical path</b> reserved for the whole session                    | No dedicated path; <b>whole message</b> hops node to node                 | No dedicated path; <b>message split into packets</b> , each routed |
| Setup phase            | <b>Required</b> (call setup) — three phases: <b>setup, data transfer, teardown</b> | None                                                                      | None (datagram) / required (virtual circuit)                       |
| Store and forward      | <b>No</b> — data flows continuously                                                | <b>Yes</b> — each node stores the <b>entire message</b> before forwarding | <b>Yes</b> , but only a <b>small packet</b> at a time              |
| Bandwidth              | <b>Reserved, wasted when idle</b>                                                  | Shared                                                                    | <b>Shared efficiently</b> (statistical multiplexing)               |
| Delay                  | <b>Setup delay high; then constant, minimal, no jitter</b>                         | <b>Very high</b> — proportional to message size at every hop              | Low; variable ( <b>jitter</b> )                                    |
| Storage at nodes       | None needed                                                                        | <b>Large</b> — must hold the whole message                                | Small buffers                                                      |
| Reliability on failure | <b>Call drops</b> — must redial                                                    | Message queued/retried                                                    | <b>Reroutes automatically</b> around the failure                   |
| Order of arrival       | Always in order                                                                    | In order                                                                  | <b>May arrive out of order</b> (datagram)                          |
| Efficiency             | <b>Poor</b> for bursty data                                                        | Better                                                                    | <b>Best</b> for bursty data                                        |
| Charging               | By <b>time</b>                                                                     | By message                                                                | By <b>volume</b>                                                   |
| Example                | <b>Telephone network (PSTN)</b> , ISDN                                             | <b>Telegram</b> , old telex, <b>email store-and-forward</b>               | <b>The Internet (IP)</b> , X.25, Frame Relay, ATM                  |

**The key insight to state:** circuit switching is optimal for **continuous, steady traffic**

like voice, because the constant reserved rate matches the source and there is no jitter.

Packet switching is optimal for **bursty data traffic**, because a data terminal is idle most of the time and reserving capacity for it is waste. This is the whole argument, and it earns the mark.

## Packet switching — datagram vs virtual circuit

|                      | Datagram approach (connectionless)                         | Virtual circuit approach (connection-oriented)                                  |
|----------------------|------------------------------------------------------------|---------------------------------------------------------------------------------|
| Setup                | <b>None</b>                                                | <b>Required</b> — a setup packet establishes the route and assigns a <b>VCI</b> |
| Routing decision     | Made <b>independently for every packet</b>                 | Made <b>once</b> , at setup; all packets follow it                              |
| Addressing in header | <b>Full source and destination address</b> in every packet | Short <b>virtual circuit identifier (VCI)</b> only                              |
| Path                 | Packets may take <b>different paths</b>                    | <b>All packets take the same path</b>                                           |
| Ordering             | <b>May arrive out of order</b>                             | <b>Always in order</b>                                                          |
| State in routers     | <b>Stateless</b>                                           | Routers hold <b>per-circuit state</b> in a table                                |
| On router failure    | Other packets rerouted; only in-flight ones lost           | <b>The whole circuit fails</b> and must be re-established                       |
| Congestion control   | Difficult                                                  | Easier — resources can be reserved at setup                                     |
| Example              | <b>IP</b>                                                  | <b>X.25, Frame Relay, ATM, MPLS</b>                                             |

## MCQ traps

| Trap                                                     | Correct answer                                                        |
|----------------------------------------------------------|-----------------------------------------------------------------------|
| "Telephone network uses?"                                | <b>Circuit switching</b> .                                            |
| "The Internet uses?"                                     | <b>Packet switching</b> (datagram).                                   |
| "Which needs the largest storage at intermediate nodes?" | <b>Message switching</b> .                                            |
| "Three phases of circuit switching?"                     | <b>Setup, data transfer, teardown</b> .                               |
| "Which guarantees in-order delivery?"                    | <b>Circuit switching</b> and <b>virtual circuits</b> ; not datagrams. |
| "Which is most efficient for bursty traffic?"            | <b>Packet switching</b> .                                             |

## 13. Multiplexing

### Concept

A single high-capacity link is far cheaper per bit than many low-capacity ones. **Multiplexing** is the set of techniques for letting several signals share one link. A **MUX** combines  $n$  inputs onto one link at the sending end; a **DEMUX** separates them at the far end. The question is always the same — *what do we divide up?* — and the three answers are **frequency**, **time** and **wavelength**.

## The techniques

|                  | FDM (Frequency Division)                                                     | TDM (Time Division)                                        | WDM (Wavelength Division)                                      |
|------------------|------------------------------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------|
| Divides          | Frequency/bandwidth                                                          | Time                                                       | Wavelength (colour) of light                                   |
| Signal type      | Analogue                                                                     | Digital (usually)                                          | Optical                                                        |
| Each user gets   | A <b>different frequency band, all of the time</b>                           | The <b>whole bandwidth, for a slot of time</b>             | A <b>different wavelength</b> , all the time                   |
| Separator needed | <b>Guard bands</b> between channels, to prevent overlap/crosstalk            | <b>Guard time / synchronisation bits</b> between slots     | Different $\lambda$ , separated by a prism/diffraction grating |
| Medium           | Coax, radio, satellite                                                       | Copper, fibre, T1/E1                                       | <b>Optical fibre only</b>                                      |
| Examples         | <b>AM/FM radio, broadcast TV, cable TV, first-generation cellular (AMPS)</b> | <b>T1/E1 carriers, digital telephony, ISDN, GSM, SONET</b> | <b>DWDM/CWDM in fibre backbones</b>                            |

**WDM is conceptually FDM applied to light** — different wavelengths *are* different frequencies. Say that; it is the connecting insight.

## Synchronous vs statistical TDM

|                  | Synchronous TDM                                              | Statistical (asynchronous) TDM                            |
|------------------|--------------------------------------------------------------|-----------------------------------------------------------|
| Slot allocation  | <b>Fixed</b> — each input has a reserved slot in every frame | <b>On demand</b> — slots go only to inputs that have data |
| Idle input       | Its slot is transmitted <b>empty — wasted bandwidth</b>      | Its slot is <b>given to someone else</b>                  |
| Addressing       | None needed (position identifies the source)                 | Each slot must carry an <b>address</b>                    |
| Number of inputs | Cannot exceed slots per frame                                | <b>Can exceed</b> — oversubscription is allowed           |
| Efficiency       | Lower                                                        | <b>Higher</b>                                             |

**Worked TDM example.** Four channels each producing **1000 bps** are multiplexed by synchronous TDM with **1 bit per slot**. Then:

- Frame = 4 bits (one slot per channel).
- Each channel sends 1000 bits/s, so **1000 frames per second**.
- **Link rate =  $4 \times 1000 = 4000$  bps**; frame duration = 1 ms; slot duration = 0.25 ms.

If instead each slot carries **4 bits** (a character), the frame is 16 bits, there are 250 frames/s, and the link rate is still 4000 bps.

**T1 carrier — the standard numerical to know.** A T1 line multiplexes **24 voice channels**.

Each channel is sampled 8000 times per second at 8 bits per sample → 64 kbps (a **DS0**). One frame =  $24 \times 8 = 192$  bits + **1 framing bit = 193 bits**, sent 8000 times a second →  **$193 \times 8000 = 1.544$  Mbps**. The European **E1** carries **32 channels** (30 voice + 2 signalling) →  $32 \times 64$  kbps = **2.048 Mbps**.

**Other access techniques** worth naming: **CDMA** (Code Division Multiple Access — all users transmit over the whole band at the same time, separated by orthogonal codes; used in 3G), **OFDM** (used in Wi-Fi, LTE, 5G), and the multiple-access protocols **ALOHA**, **slotted ALOHA**, **CSMA/CD** (Ethernet — listen, transmit, detect collision, jam, back off exponentially) and **CSMA/CA** (Wi-Fi — collision *avoidance* with RTS/CTS, because a wireless node cannot listen while transmitting and suffers the **hidden terminal problem**).

## MCQ traps

| Trap                                    | Correct answer                                                                |
|-----------------------------------------|-------------------------------------------------------------------------------|
| "Which multiplexing needs guard bands?" | <b>FDM</b> .                                                                  |
| "WDM is used with?"                     | <b>Optical fibre</b> .                                                        |
| "T1 rate and channels?"                 | <b>1.544 Mbps</b> , <b>24</b> channels. E1 = <b>2.048 Mbps</b> , 32 channels. |
| "One voice channel (DS0)?"              | <b>64 kbps</b> (8000 samples/s $\times$ 8 bits).                              |
| "Which TDM wastes idle slots?"          | <b>Synchronous</b> TDM.                                                       |
| "Ethernet access method?"               | <b>CSMA/CD</b> . Wi-Fi = <b>CSMA/CA</b> .                                     |

## 14. Channel capacity — Nyquist and Shannon

### Concept

Two different questions, two different formulas, and telling them apart is most of the marks:

- **Nyquist** answers: *given a perfectly clean (noiseless) channel of bandwidth  $B$ , how fast can I signal?* The limit comes purely from bandwidth and from how many distinguishable **signal levels** you use.
- **Shannon** answers: *given a real channel with noise, what is the absolute maximum, no matter how clever my encoding?* Here the limit comes from bandwidth and the **signal-to-noise ratio**.

### Nyquist (noiseless channel):

$$C = 2 \times B \times \log_2(L) \text{ bits per second}$$

where  $B$  = bandwidth in Hz,  $L$  = number of discrete signal levels. (The maximum **signalling rate** is  **$2B$  baud**; each symbol carries  $\log_2 L$  bits.)

### Shannon (noisy channel):

$$C = B \times \log_2(1 + S/N) \text{ bits per second}$$

where S/N is the signal-to-noise **power ratio** (a plain number, not decibels).

**Decibel conversion — always needed:**

$$\text{SNR(dB)} = 10 \log_{10}(S/N), \text{ so } S/N = 10^{(\text{SNR(dB)}/10)}$$

Useful values: 10 dB → 10; 20 dB → 100; **30 dB → 1000**; 36 dB → ~3981; 40 dB → 10,000.

**Bit rate vs baud rate** (a guaranteed MCQ):

**bit rate = baud rate × log<sub>2</sub>(L)** — that is, bits per second = symbols per second × bits per symbol. Bit rate ≥ baud rate always; they are equal only when L = 2.

### Worked example 1 — Nyquist, binary

A noiseless channel has bandwidth **3000 Hz**. Find the capacity for **2 levels**.

$$C = 2 \times 3000 \times \log_2 2 = 2 \times 3000 \times 1 = \mathbf{6000 \text{ bps} = 6 \text{ kbps}}$$

### Worked example 2 — Nyquist, multilevel

Same 3000 Hz channel, but **4 signal levels**.

$$C = 2 \times 3000 \times \log_2 4 = 2 \times 3000 \times 2 = \mathbf{12,000 \text{ bps} = 12 \text{ kbps}}$$

$$\text{With } \mathbf{16 \text{ levels}}: C = 2 \times 3000 \times \log_2 16 = 2 \times 3000 \times 4 = \mathbf{24 \text{ kbps}}.$$

**Comment worth writing:** doubling the levels adds only **one bit per symbol**, so capacity grows **logarithmically** with L, while noise immunity falls (levels get closer together). This is why you cannot simply increase L forever — and Shannon tells you exactly where the wall is.

### Worked example 3 — Shannon, the telephone line

A telephone line has bandwidth **3000 Hz** and **SNR = 30 dB**. Find the maximum capacity.

$$\mathbf{\text{Step 1 — convert dB.}} \quad S/N = 10^{(30/10)} = 10^3 = \mathbf{1000}$$

**Step 2 — apply Shannon.**

$$C = 3000 \times \log_2(1 + 1000) = 3000 \times \log_2(1001)$$

$$\log_2(1001) = \ln(1001)/\ln(2) = 6.9088/0.6931 \approx \mathbf{9.968}$$

$$C = 3000 \times 9.968 \approx \mathbf{29,904 \text{ bps} \approx 30 \text{ kbps}}$$

**Comment:** this is precisely why dial-up modems plateaued around **28.8–33.6 kbps** — they were pressing against the Shannon limit of the analogue local loop. (56 k modems beat it only by having one end fully digital, avoiding one analogue-to-digital conversion.) Mentioning this turns a formula into an explanation and reliably earns extra credit.

### Worked example 4 — Shannon with a low SNR

A channel of bandwidth **1 MHz** has **S/N = 63**.

$$C = 10^6 \times \log_2(1 + 63) = 10^6 \times \log_2 64 = 10^6 \times 6 = \mathbf{6 \text{ Mbps}}$$

### Worked example 5 — combining Shannon and Nyquist (the classic two-part

## question)

"A channel has a bandwidth of 1 MHz and an SNR of 63. What is the appropriate bit rate and how many signal levels are required?"

### Step 1 — Shannon gives the upper bound.

$C = 10^6 \times \log_2(1 + 63) = 10^6 \times 6 = 6 \text{ Mbps}$ . This is the ceiling; we choose a bit rate at or below it — take **6 Mbps**.

### Step 2 — Nyquist tells us how many levels achieve it.

$$6 \times 10^6 = 2 \times 10^6 \times \log_2 L$$

$$\rightarrow \log_2 L = 3$$

$$\rightarrow L = 8 \text{ signal levels}$$

**Answer: 6 Mbps using 8 signal levels.**

*The method to state: Shannon fixes the limit, Nyquist tells you the signalling scheme.*

## Worked example 6 — SNR needed for a target rate

What SNR (in dB) is needed to achieve 100 kbps over a 10 kHz channel?

$$100,000 = 10,000 \times \log_2(1 + S/N)$$

$$\rightarrow \log_2(1 + S/N) = 10$$

$$\rightarrow 1 + S/N = 2^{10} = 1024$$

$$\rightarrow S/N = 1023$$

$$\rightarrow \text{SNR(dB)} = 10 \log_{10}(1023) = 10 \times 3.0099 \approx 30.1 \text{ dB}$$

## Worked example 7 — bit rate vs baud rate

A signal carries 4 bits per symbol at 2000 baud. Find the bit rate. If the bit rate is 12,000 bps at 3000 baud, how many levels are used?

(a) Bit rate = baud  $\times$  bits/symbol =  $2000 \times 4 = 8000 \text{ bps}$

(b) bits/symbol =  $12,000 / 3000 = 4 \rightarrow L = 2^4 = 16 \text{ levels}$

## Worked example 8 — Nyquist sampling theorem (different but related)

Do not confuse Nyquist's *capacity* formula with the **Nyquist sampling theorem**: to digitise an analogue signal without loss, sample at **at least twice the highest frequency** ( $f_s \geq 2 f_{\max}$ ). For voice band-limited to 4 kHz:  $f_s = 8000 \text{ samples/s}$ ; at 8 bits per sample this gives **64 kbps** — the DS0 channel of §13. Under-sampling causes **aliasing**.

## Likely exam questions

- [5] State Nyquist's and Shannon's formulas. What does each account for?
- [5] Differentiate between bit rate and baud rate with an example.
- [5] Calculate the capacity of a 3 kHz noiseless channel using 8 signal levels.
- [5] A channel has bandwidth 4 kHz and SNR 20 dB. Find its capacity.  
( $S/N = 100$ ;  $C = 4000 \times \log_2 101 = 4000 \times 6.658 \approx 26.6 \text{ kbps}$ )
- [10] Explain channel capacity. Derive/state the Nyquist and Shannon formulas and solve: a channel of 1 MHz bandwidth with SNR 63 — find the maximum bit rate and the number of

signal levels required.

- **[10]** What factors limit the data rate of a channel? Explain bandwidth, noise, attenuation and distortion, and illustrate with Nyquist and Shannon calculations.

## MCQ traps

| Trap                                     | Correct answer                                      |
|------------------------------------------|-----------------------------------------------------|
| "Which formula includes noise?"          | <b>Shannon</b> . Nyquist assumes <b>noiseless</b> . |
| "Nyquist for 2 levels, B = 4 kHz?"       | $2 \times 4000 \times 1 =$ <b>8 kbps</b> .          |
| "SNR 30 dB = ?"                          | <b>1000</b> (not 30).                               |
| "Is bit rate always $\geq$ baud rate?"   | <b>Yes</b> ; equal only when L = 2.                 |
| "Nyquist sampling rate for 4 kHz voice?" | <b>8000 samples/s</b> .                             |
| "Unit of bandwidth vs data rate?"        | <b>Hz vs bps</b> .                                  |

## 15. DNS — Domain Name System

### Concept

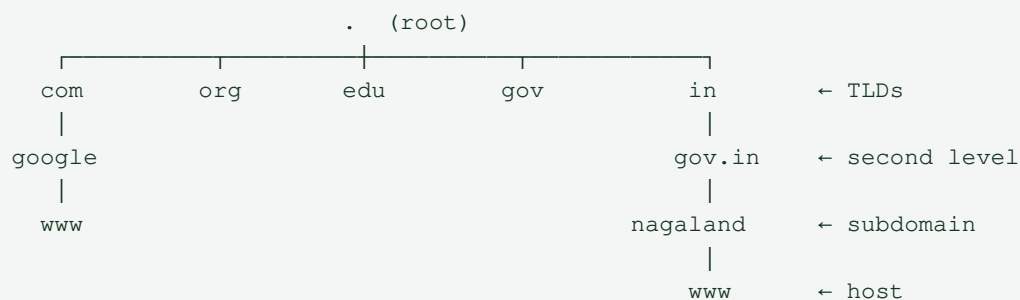
People remember `www.nagaland.gov.in`; routers only understand `104.18.32.7`. **DNS is the distributed database that translates between them.** Two design decisions define it:

1. **It is hierarchical**, so the name space can be delegated. Nobody has to know every name.
2. **It is distributed**, so no single server holds everything and no single failure is fatal.

### The name space

An inverted tree again. Read a domain name **right to left**, from most general to most specific. A fully qualified domain name (FQDN) technically ends with a dot for the root:

`www.nagaland.gov.in.`



| TLD type | Examples |
|----------|----------|
|----------|----------|

|                                 |                                                       |
|---------------------------------|-------------------------------------------------------|
| <b>gTLD</b> (generic)           | .com, .org, .net, .edu, .gov, .mil, .int, .info, .biz |
| <b>ccTLD</b> (country code)     | .in, .uk, .us, .jp, .cn                               |
| <b>Second level under ccTLD</b> | .co.in, .gov.in, .ac.in, .nic.in                      |

A **zone** is the portion of the name space an individual server is **authoritative** for — the unit of administrative delegation. A **domain** is a subtree of the name space. They coincide unless the domain delegates sub-zones.

## Types of name server

| Type                                         | Role                                                                                                                                                                                                                          |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Root name server</b>                      | Top of the hierarchy; knows the authoritative servers for every TLD. There are <b>13 logical root server addresses</b> (A–M), served by hundreds of physical machines via anycast                                             |
| <b>TLD server</b>                            | Authoritative for a top-level domain; knows the servers for each second-level domain                                                                                                                                          |
| <b>Authoritative server</b>                  | Holds the <b>actual zone file</b> with the real records for a domain. <b>Primary/master</b> holds the editable copy; <b>secondary/slave</b> copies it via a <b>zone transfer</b> (AXFR/IXFR, over <b>TCP</b> ) for redundancy |
| <b>Recursive resolver / local DNS server</b> | The server your machine is configured to ask (usually your ISP's or 8.8.8.8). It does the legwork on your behalf and <b>caches</b> results                                                                                    |
| <b>Caching-only server</b>                   | Holds no zone of its own; just resolves and caches                                                                                                                                                                            |

## Resolution — the process to describe

**Recursive query:** the client asks its local resolver and demands a *final answer* — "either the address or an error, but do not send me elsewhere." The resolver takes on all the work.

**Iterative query:** the resolver asks a server, which replies "I don't know, but ask *this* server" — a **referral**. The resolver then asks the next one, and so on.

**Worked resolution of `www.example.com` (write this out step by step for 10 marks):**

1. The browser checks its own cache, then the OS cache, then the local **hosts file** (`/etc/hosts`, or `C:\Windows\System32\drivers\etc\hosts`). Hit → done.
2. Miss → the host sends a **recursive** query to its configured **local DNS resolver**.
3. The resolver checks **its** cache. Miss → it queries a **root** server **iteratively**.
4. The root replies with a referral to the **.com TLD** servers.
5. The resolver queries a **.com** server → referral to the **authoritative** servers for `example.com`.
6. The resolver queries the authoritative server → it returns the **A record**:  
`www.example.com → 93.184.216.34`.
7. The resolver **caches** the answer for its **TTL** and returns it to the client.

DNS uses **UDP port 53** for queries (small, fast, retried if lost) and **TCP port 53** for **zone transfers** and responses larger than 512 bytes.

## Resource records — explicitly named in your Forensic syllabus

A resource record is a five-tuple: **Domain name, TTL, Class (IN), Type, Value**.

| Type         | Name                 | Purpose                                                                                                                                                     | Example                                                          |
|--------------|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| <b>A</b>     | Address              | Maps a name to an <b>IPv4</b> address                                                                                                                       | <code>www.example.com. 3600 IN A 93.184.216.34</code>            |
| <b>AAAA</b>  | Quad-A               | Maps a name to an <b>IPv6</b> address                                                                                                                       | <code>IN AAAA 2606:2800:220:1::1</code>                          |
| <b>NS</b>    | Name Server          | Names the <b>authoritative</b> servers for the zone                                                                                                         | <code>example.com. IN NS ns1.example.com.</code>                 |
| <b>MX</b>    | Mail Exchange        | The <b>mail server</b> for the domain, with a <b>preference number (lower = higher priority)</b>                                                            | <code>example.com. IN MX 10 mail.example.com.</code>             |
| <b>CNAME</b> | Canonical Name       | An <b>alias</b> pointing to another name                                                                                                                    | <code>ftp.example.com. IN CNAME www.example.com.</code>          |
| <b>PTR</b>   | Pointer              | <b>Reverse lookup</b> — IP → name, in the <code>in-addr.arpa</code> domain                                                                                  | <code>34.216.184.93.in-addr.arpa. IN PTR www.example.com.</code> |
| <b>SOA</b>   | Start of Authority   | Zone parameters: primary server, admin email, <b>serial number</b> , refresh, retry, expire, minimum TTL. <b>Exactly one per zone, and it must be first</b> |                                                                  |
| <b>TXT</b>   | Text                 | Arbitrary text — used for <b>SPF, DKIM, DMARC</b> (email anti-spoofing) and domain verification                                                             |                                                                  |
| <b>SRV</b>   | Service              | Locates the host/port for a named service                                                                                                                   | <code>_sip._tcp.example.com.</code>                              |
| <b>HINFO</b> | Host Info            | CPU and OS description (rarely used; an information-disclosure risk)                                                                                        |                                                                  |
| <b>CAA</b>   | Cert Authority Auth. | Which CAs may issue certificates for the domain                                                                                                             |                                                                  |

**For the Forensic paper**, note the investigative tools: `nslookup`, `dig`, `host`, `whois` (registrant details, registration and expiry dates, name servers), and reverse DNS via PTR. TXT records holding **SPF** and **DKIM** are what you check when analysing whether an email was spoofed.

## Likely exam questions

- [5] What is DNS? Why is it needed?

- **[5]** Explain the hierarchical structure of the DNS name space with a diagram.
- **[5]** Differentiate between recursive and iterative DNS queries.
- **[5]** What are resource records? Explain A, MX, CNAME, NS and PTR.
- **[10]** Explain the Domain Name System: the name space, zones and domains, types of name server, resource records, and the complete resolution of `www.example.com` step by step.
- **[15]** Discuss DNS in detail — architecture, name servers, resource records, caching and TTL, recursive vs iterative resolution — and explain DNS-based attacks (cache poisoning, spoofing, tunnelling) and their countermeasures.

## MCQ traps

| Trap                                                     | Correct answer                                                     |
|----------------------------------------------------------|--------------------------------------------------------------------|
| "DNS port and transport?"                                | <b>53</b> , <b>UDP</b> for queries, <b>TCP</b> for zone transfers. |
| "Record for IPv6?"                                       | <b>AAAA</b> (A is IPv4).                                           |
| "Record for a mail server?"                              | <b>MX</b> . For an alias → <b>CNAME</b> .                          |
| "Reverse lookup record?"                                 | <b>PTR</b> , under <code>in-addr.arpa</code> .                     |
| "How many SOA records per zone?"                         | <b>Exactly one</b> .                                               |
| "Lower MX preference means?"                             | <b>Higher</b> priority.                                            |
| "Number of root server addresses?"                       | <b>13</b> (A–M).                                                   |
| "Which query type does the client send to its resolver?" | <b>Recursive</b> .                                                 |

## 16. Electronic mail

### Concept

Email is a **store-and-forward** system, and that single phrase explains its architecture: the sender and recipient are almost never online at the same time, so the message is handed to a server, which holds it, forwards it, and eventually parks it in a mailbox until the recipient comes to collect.

### Email architecture — the components

| Component       | Full name                | Role                                                              | Examples                                                   |
|-----------------|--------------------------|-------------------------------------------------------------------|------------------------------------------------------------|
| <b>UA / MUA</b> | (Mail) <b>User Agent</b> | The program the human uses to compose, send, read and manage mail | Outlook, Thunderbird, Gmail web UI, <code>mail/mutt</code> |
| <b>MSA</b>      | Mail Submission Agent    | Accepts mail from the UA (port 587), authenticates the sender     |                                                            |

|                            |                                  |                                              |
|----------------------------|----------------------------------|----------------------------------------------|
| Server-side search / flags | No                               | <b>Yes</b> (read/unread, flagged, answered)  |
| Complexity / bandwidth     | Simple, low                      | Complex, higher                              |
| Best for                   | One device, limited server quota | Multiple devices — <b>the modern default</b> |

## MIME

SMTP was defined for **7-bit US-ASCII text**. It cannot carry an image, a PDF, a video, or even a message in a non-Latin script. **MIME solves this without changing SMTP**: it **encodes** binary content into ASCII at the sender and decodes it at the receiver, and it adds headers describing what the content is.

### The five MIME headers:

| Header                                  | Purpose                                                                                                                                                                                                                                            |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>MIME-Version:</code>              | 1.0                                                                                                                                                                                                                                                |
| <code>Content-Type:</code>              | Type/subtype — <code>text/plain</code> , <code>text/html</code> , <code>image/jpeg</code> , <code>application/pdf</code> , <b><code>multipart/mixed</code></b> (message + attachments), <code>multipart/alternative</code> (plain + HTML versions) |
| <code>Content-Transfer-Encoding:</code> | How the data was encoded: <b><code>base64</code></b> (binary → ASCII, expands by ~33%), <b><code>quoted-printable</code></b> (mostly-text with a few special characters), <code>7bit</code> , <code>8bit</code> , <code>binary</code>              |
| <code>Content-Id:</code>                | Unique identifier for the part                                                                                                                                                                                                                     |
| <code>Content-Description:</code>       | Human-readable description                                                                                                                                                                                                                         |

`multipart/mixed` uses a **boundary string** to separate the parts.

## Email headers and tracing — your Forensic paper's TP-II §2

An email has **headers** and a **body**. Headers are added at each hop, and — crucially — **new Received: headers are prepended at the top**, so **you read them from the bottom upwards** to trace the message's actual journey from origin to destination.

| Header                        | Meaning / forensic value                                                                                                                                                                                                                                                                                                                                |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b><code>Received:</code></b> | Added by <b>each MTA</b> : <code>from &lt;claimed host&gt; by &lt;receiving host&gt; with &lt;protocol&gt;; &lt;timestamp&gt;</code> . <b>Read bottom-up</b> . The <b>lowest</b> Received header is the origin — and is the least trustworthy, since a sender can forge earlier ones. The <b>topmost</b> ones, added by servers you trust, are reliable |

|                                |                                                                                                                                         |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>From:</b>                   | <b>Trivially forged</b> — it is just text supplied by the sender. Compare it against the SMTP envelope <b>MAIL FROM</b> and against SPF |
| <b>Return-Path:</b>            | The <b>envelope sender</b> ; where bounces go. A mismatch with <b>From:</b> is a spoofing indicator                                     |
| <b>Reply-To:</b>               | Where replies go — a classic phishing trick is a legitimate-looking <b>From:</b> with an attacker's <b>Reply-To:</b>                    |
| <b>Message-ID:</b>             | Unique identifier assigned by the originating server; its domain part reveals the true origin                                           |
| <b>X-Originating-IP:</b>       | The client's IP, added by some webmail providers                                                                                        |
| <b>Authentication-Results:</b> | <b>SPF / DKIM / DMARC</b> verdicts — <b>pass, fail, softfail</b>                                                                        |
| <b>X-Mailer: / User-Agent:</b> | Sending software — bulk mailers and scripts often reveal themselves here                                                                |

**The anti-spoofing trio** — name all three:

- **SPF** (Sender Policy Framework): a **TXT** record listing which IPs may send mail for the domain. Verifies the **envelope sender's IP**.
- **DKIM** (DomainKeys Identified Mail): the sending server **digitally signs** headers and body with a private key; the public key is published in **DNS**. Verifies **integrity and origin**.
- **DMARC**: a policy record saying what to do (**none/quarantine/reject**) when SPF or DKIM fails, and requiring **alignment** with the **From:** domain. Sends reports back.

**Why email spoofing is so easy** — the sentence to write: **SMTP was designed in 1982 with no authentication whatsoever**; anyone who can connect to port 25 can claim to be anyone. SPF, DKIM and DMARC are all bolted on afterwards, via DNS.

## Likely exam questions

- **[5]** Explain the architecture of an email system (UA, MTA, MDA).
- **[5]** Differentiate between POP3 and IMAP.
- **[5]** What is MIME? Why is it needed? Name its headers.
- **[5]** Explain SMTP and list its commands.
- **[5]** How would you trace the origin of an email from its headers?
- **[10]** Explain electronic mail architecture and the protocols involved — SMTP, POP3, IMAP and MIME — with a diagram, stating the port numbers and the direction of each.
- **[15]** Discuss email in detail: architecture, message format (envelope, header, body), SMTP, MIME encoding, retrieval by POP3 vs IMAP, and email spoofing — how it is done, how headers are analysed to detect it, and how SPF, DKIM and DMARC prevent it.

## MCQ traps

| Trap                                    | Correct answer                                                                   |
|-----------------------------------------|----------------------------------------------------------------------------------|
| "SMTP port?"                            | <b>25</b> . POP3 <b>110</b> , IMAP <b>143</b> , HTTPS 443.                       |
| "Which protocol <i>sends</i> mail?"     | <b>SMTP</b> (push). POP3/IMAP retrieve.                                          |
| "Which keeps mail on the server?"       | <b>IMAP</b> .                                                                    |
| "What does MIME do?"                    | Lets SMTP carry <b>non-ASCII/binary</b> data — it does <b>not</b> transfer mail. |
| "Base64 expands data by?"               | About <b>33%</b> (3 bytes → 4 characters).                                       |
| "Read Received headers in which order?" | <b>Bottom to top</b> .                                                           |
| "Is the <b>From:</b> header reliable?"  | <b>No</b> — easily forged.                                                       |
| "Which DNS record supports SPF?"        | <b>TXT</b> .                                                                     |

## 17. HTTP and the World Wide Web

### Concept

The **World Wide Web** is not the Internet — it is one *application* running on it, invented by **Tim Berners-Lee at CERN in 1989–91**. It rests on three ideas: a **naming scheme** (URL), a **transfer protocol** (HTTP) and a **markup language** (HTML), tied together by **hypertext links**.

### URL structure

```
https :// www.example.com : 443 /docs/index.html ?id=5&sort=asc #section2
|         |           |         |           |           |
scheme  host      port   path     query    fragment
```

| Part                   | Meaning                                                                        |
|------------------------|--------------------------------------------------------------------------------|
| <b>Scheme/protocol</b> | <b>http</b> , <b>https</b> , <b>ftp</b> , <b>mailto</b> , <b>file</b>          |
| <b>Host</b>            | Domain name or IP                                                              |
| <b>Port</b>            | Optional; defaults to 80 for HTTP, <b>443</b> for HTTPS                        |
| <b>Path</b>            | Resource location on the server                                                |
| <b>Query string</b>    | <b>?key=value&amp;key=value</b> — parameters                                   |
| <b>Fragment</b>        | <b>#anchor</b> — handled by the <b>browser only</b> , never sent to the server |

## HTTP

**HTTP (HyperText Transfer Protocol)** runs over **TCP port 80**; **HTTPS** is HTTP inside a **TLS/SSL** tunnel on **port 443**. It is a **request–response, stateless, client–server** protocol.

**Stateless** means the server retains no memory of previous requests. That is deliberate — it makes servers massively scalable — but it breaks shopping carts and logins, so state is reintroduced by **cookies** (small key–value pairs the server sets via **Set-Cookie:** and the browser returns in every subsequent **Cookie:** header), plus **sessions**, hidden form fields and URL rewriting. Note for your Forensic paper: **cookies, cache, history and bookmarks are exactly the browser artefacts named in TP-II §2.**

**HTTP methods:**

| Method         | Purpose                                                                                           | Safe? | Idempotent? |
|----------------|---------------------------------------------------------------------------------------------------|-------|-------------|
| <b>GET</b>     | Retrieve a resource. Parameters in the <b>URL</b> — visible, logged, bookmarkable, length-limited | Yes   | Yes         |
| <b>POST</b>    | Submit data. Parameters in the <b>body</b> — not in the URL, no length limit                      | No    | No          |
| <b>HEAD</b>    | Like GET but returns <b>headers only</b> , no body                                                | Yes   | Yes         |
| <b>PUT</b>     | Upload/replace a resource                                                                         | No    | Yes         |
| <b>DELETE</b>  | Remove a resource                                                                                 | No    | Yes         |
| <b>OPTIONS</b> | List the methods the server supports                                                              | Yes   | Yes         |
| <b>CONNECT</b> | Establish a tunnel (used by proxies for HTTPS)                                                    | No    | No          |
| <b>TRACE</b>   | Echo the request back (diagnostic)                                                                | Yes   | Yes         |

**GET vs POST is a standard 5-marker:** GET puts data in the query string (visible in the URL, in browser history and in server logs — so **never use it for passwords**), is limited in length, can be cached and bookmarked, and should not change server state. POST puts data in the body, has no practical size limit, is not cached or bookmarked, and is used for state-changing submissions.

**Status codes** — memorise the classes and the famous members:

| Class      | Meaning            | Examples                                                                         |
|------------|--------------------|----------------------------------------------------------------------------------|
| <b>1xx</b> | Informational      | 100 Continue, 101 Switching Protocols                                            |
| <b>2xx</b> | <b>Success</b>     | <b>200 OK</b> , 201 Created, 204 No Content                                      |
| <b>3xx</b> | <b>Redirection</b> | 301 Moved Permanently, 302 Found, <b>304 Not Modified</b> (cached copy is valid) |

|            |                     |                                                                                                                                        |
|------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>4xx</b> | <b>Client error</b> | 400 Bad Request, <b>401 Unauthorized</b> , <b>403 Forbidden</b> , <b>404 Not Found</b> , 405 Method Not Allowed, 429 Too Many Requests |
| <b>5xx</b> | <b>Server error</b> | <b>500 Internal Server Error</b> , 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout                                       |

**401 vs 403** is a classic MCQ: **401 = not authenticated** (you have not proved who you are); **403 = authenticated but not authorised** (we know who you are and you still may not).

**HTTP versions**: HTTP/1.0 opened a new TCP connection per object; **HTTP/1.1** added **persistent connections** (keep-alive), pipelining, host headers and caching controls; **HTTP/2** added binary framing, multiplexing and header compression; **HTTP/3** runs over **QUIC/UDP**.

**Other web/Internet services** (Diploma P-I §6): **FTP** (20/21, separate control and data connections; active vs passive mode), **TFTP** (69, UDP, no authentication), **Telnet** (23, **plaintext — insecure**), **SSH** (22, encrypted replacement), **chat/IRC**, **bulletin boards/NNTP**, **video conferencing** (H.323, SIP, RTP), **search engines** (crawler, indexer, query processor, ranking), **ISP connection types**:

| Access type       | Speed              | Notes                                                                 |
|-------------------|--------------------|-----------------------------------------------------------------------|
| Dial-up           | ≤ 56 kbps          | Uses the voice line via a modem; <b>ties up the phone</b> ; obsolete  |
| Leased line       | 64 kbps – Gbps     | <b>Dedicated, always-on</b> , symmetric, expensive; for organisations |
| ISDN              | 64/128 kbps (BRI)  | Digital over copper                                                   |
| DSL/ADSL          | 1–100 Mbps         | Asymmetric; shares the phone line but does not block it               |
| Cable             | 10–1000 Mbps       | Shared coax from the cable TV plant                                   |
| VSAT              | 64 kbps – few Mbps | Satellite; remote areas; high latency                                 |
| Fibre (FTTH)      | 100 Mbps – 1 Gbps+ | Highest performance                                                   |
| Mobile (3G/4G/5G) | 1 Mbps – 1 Gbps    | Wireless, mobile                                                      |

## MCQ traps

| Trap                                     | Correct answer                                                |
|------------------------------------------|---------------------------------------------------------------|
| "HTTP port / HTTPS port?"                | <b>80 / 443</b> .                                             |
| "HTTP is stateless — how is state kept?" | <b>Cookies</b> / sessions.                                    |
| "404 / 500 / 403 / 401?"                 | Not Found / Internal Server Error / Forbidden / Unauthorized. |
| "Which method hides data from the URL?"  | <b>POST</b> .                                                 |

|                                        |                                                                |
|----------------------------------------|----------------------------------------------------------------|
| "Who invented the WWW?"                | <b>Tim Berners-Lee</b> , CERN, 1989–91.                        |
| "Is the Web the same as the Internet?" | <b>No</b> — the Web is an application running on the Internet. |
| "FTP ports?"                           | <b>21 control, 20 data</b> .                                   |
| "Telnet is secure?"                    | <b>No</b> — plaintext; use <b>SSH</b> .                        |

## 18. Network security, cryptography and firewalls

### Concept

Security is usually framed as the **CIA triad** plus two:

| Goal                   | Meaning                                   | Threatened by                              |
|------------------------|-------------------------------------------|--------------------------------------------|
| <b>Confidentiality</b> | Only authorised parties can read the data | Eavesdropping, sniffing, traffic analysis  |
| <b>Integrity</b>       | Data is not altered in transit or storage | Modification, replay, MITM                 |
| <b>Availability</b>    | Services are usable when needed           | <b>DoS / DDoS</b> , ransomware             |
| <b>Authentication</b>  | Parties are who they claim to be          | Spoofing, masquerade                       |
| <b>Non-repudiation</b> | A sender cannot later deny having sent    | — (provided by <b>digital signatures</b> ) |

**Attack classification** — a reliable 5-marker:

|               | Passive attacks                                  | Active attacks                                                                                     |
|---------------|--------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Nature        | <b>Observe</b> only; do not alter data           | <b>Alter</b> data or disrupt service                                                               |
| Detectability | <b>Very hard to detect</b> (nothing changes)     | Detectable                                                                                         |
| Prevention    | <b>Prevention</b> is the goal (encryption)       | <b>Detection and recovery</b> is the goal                                                          |
| Examples      | <b>Eavesdropping/sniffing</b> , traffic analysis | <b>Masquerade/spoofing</b> , <b>replay</b> , <b>modification</b> , <b>denial of service</b> , MITM |

**Named threats to define in one line each:**

| Threat                 | Definition                                                                                                                                                                               |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Packet sniffing</b> | Capturing traffic off the wire/air with a NIC in <b>promiscuous mode</b> (Wireshark, tcpdump). Trivial on a hub or Wi-Fi; needs <b>ARP spoofing</b> or <b>port mirroring</b> on a switch |

|                          |                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Spoofing</b>          | Forging a source identity — <b>IP spoofing</b> , <b>MAC spoofing</b> , <b>ARP spoofing</b> , <b>DNS spoofing/cache poisoning</b> , <b>email spoofing</b>                                                                                                                                                                                                      |
| <b>Man-in-the-middle</b> | Attacker relays and possibly alters traffic between two parties who believe they talk directly                                                                                                                                                                                                                                                                |
| <b>Replay</b>            | Capturing a valid message and re-sending it later. Countered by <b>nonces</b> , <b>timestamps</b> , <b>sequence numbers</b>                                                                                                                                                                                                                                   |
| <b>DoS / DDoS</b>        | Exhausting a resource so legitimate users are denied service; <b>distributed</b> when launched from a <b>botnet</b> . Examples: SYN flood, ping of death, smurf, amplification                                                                                                                                                                                |
| <b>Phishing</b>          | Fraudulent messages inducing the victim to reveal credentials; <b>spear phishing</b> is targeted                                                                                                                                                                                                                                                              |
| <b>Malware</b>           | <b>Virus</b> (attaches to a host file, needs a host and user action), <b>worm</b> (self-replicating, self-propagating over a <b>network</b> , needs <b>no host file</b> ), <b>Trojan horse</b> (disguised as legitimate, does not self-replicate), <b>ransomware</b> , <b>spyware</b> , <b>keylogger</b> , <b>rootkit</b> , <b>botnet</b> , <b>logic bomb</b> |
| <b>SQL injection</b>     | Injecting SQL through an input field (see the DBMS file)                                                                                                                                                                                                                                                                                                      |
| <b>XSS</b>               | Injecting script into a web page viewed by others                                                                                                                                                                                                                                                                                                             |

**Virus vs worm** is asked constantly: a **virus requires a host file and human action to spread**; a **worm is standalone and spreads by itself across a network**.

## Cryptography

**Encryption** turns **plaintext** into **ciphertext** using an **algorithm (cipher)** and a **key**; **decryption** reverses it. **Kerckhoffs's principle**: the security must rest entirely in the **key**, never in the secrecy of the algorithm.

### Symmetric (secret key / private key / conventional)

**One key** is used for both encryption and decryption, and both parties must have it.

- Fast — suitable for **bulk data**.
- **Two problems**: (1) **key distribution** — how do you get the key to the other party securely in the first place? (2) **key explosion** —  **$n(n-1)/2$  keys** are needed for  $n$  parties to communicate pairwise. For  $n = 100$  that is **4,950** keys.
- **Cannot provide non-repudiation** — since both parties hold the same key, either could have produced any message.
- Algorithms: **DES** (56-bit key, 64-bit block — **broken by brute force**), **3DES/Triple DES** (168/112-bit effective), **AES** (Rijndael; **128/192/256-bit keys**, 128-bit block — the current standard), **Blowfish**, **Twofish**, **IDEA**, **RC4** (stream, deprecated), **RC5**.
- Types: **block ciphers** (fixed-size blocks, modes ECB/CBC/CFB/OFB/CTR) and **stream**

**ciphers** (bit/byte at a time).

### Asymmetric (public key)

A **mathematically related key pair**: a **public key**, published to the world, and a **private key**, never disclosed. What one key encrypts, only the other can decrypt. Introduced by **Diffie and Hellman (1976)**.

The two modes — and getting these the right way round is the whole topic:

| Goal                                      | Encrypt with                     | Decrypt with                      | Why                                                                                          |
|-------------------------------------------|----------------------------------|-----------------------------------|----------------------------------------------------------------------------------------------|
| <b>Confidentiality</b>                    | The <b>receiver's PUBLIC</b> key | The receiver's <b>private</b> key | Only the receiver holds the private key, so only the receiver can read it                    |
| <b>Authentication / digital signature</b> | The <b>sender's PRIVATE</b> key  | The sender's <b>public</b> key    | Only the sender could have produced it, so it proves origin and gives <b>non-repudiation</b> |

Algorithms: **RSA** (Rivest–Shamir–Adleman, 1977; security rests on the difficulty of **factoring large primes**), **Diffie–Hellman** (key *exchange* only, not encryption; based on the **discrete logarithm** problem), **ECC** (elliptic curve — equivalent strength with much shorter keys), **DSA/EIGamal**.

Comparison table:

| Criterion        | Symmetric                             | Asymmetric                                          |
|------------------|---------------------------------------|-----------------------------------------------------|
| Keys             | <b>One shared secret</b>              | <b>Key pair</b> (public + private)                  |
| Speed            | <b>Fast</b> (100–1000× faster)        | <b>Slow</b>                                         |
| Key distribution | <b>The hard problem</b>               | <b>Solved</b> — publish the public key              |
| Keys for n users | $n(n-1)/2$                            | $2n$                                                |
| Suitable for     | <b>Bulk data encryption</b>           | <b>Key exchange, digital signatures, small data</b> |
| Non-repudiation  | <b>No</b>                             | <b>Yes</b>                                          |
| Key length       | 128–256 bits                          | 2048–4096 bits (RSA) for comparable strength        |
| Examples         | DES, 3DES, <b>AES</b> , Blowfish, RC4 | <b>RSA</b> , Diffie–Hellman, ECC, DSA               |

The hybrid system — how TLS/HTTPS and PGP actually work, and a superb 10-mark answer: use **asymmetric cryptography** to exchange a randomly generated symmetric "**session key**", then use **symmetric cryptography** for the actual data. You get the key-distribution solution of public key cryptography *and* the speed of symmetric cryptography. Say this explicitly — it shows you understand why both exist.

## Hash functions and digital signatures

A **cryptographic hash function** takes input of any length and produces a **fixed-length digest**. Properties: **one-way** (cannot invert), **deterministic**, **avalanche effect** (one bit change → completely different digest), **collision-resistant**.

| Algorithm         | Digest size             | Status                                                                               |
|-------------------|-------------------------|--------------------------------------------------------------------------------------|
| MD5               | 128 bits (32 hex chars) | <b>Broken</b> — collisions are easy. Still used for file identification in forensics |
| SHA-1             | 160 bits (40 hex chars) | <b>Broken</b> (2017) — deprecated                                                    |
| SHA-256 / SHA-512 | 256 / 512 bits          | Current standard                                                                     |
| SHA-3             | Variable                | Newest standard                                                                      |

Hashes give **integrity**, not confidentiality. In forensics, the **hash value of an image is computed before and after acquisition** and compared; matching hashes prove the evidence was not altered.

### Digital signature — the mechanism (draw this):



If **digest<sub>1</sub> = digest<sub>2</sub>**, then (a) the message was not altered — **integrity**; (b) it came from the holder of the private key — **authentication**; and (c) the sender cannot deny it — **non-repudiation**.

**Why hash the message first instead of signing it whole?** Because asymmetric encryption is slow and the message may be huge; hashing reduces it to a fixed small digest. State this.

### Digital signature vs encryption:

|                    | Digital signature                          | Encryption                   |
|--------------------|--------------------------------------------|------------------------------|
| Provides           | Authentication, integrity, non-repudiation | Confidentiality              |
| Key used to create | Sender's <b>private</b> key                | Receiver's <b>public</b> key |

|                         |                                      |                               |
|-------------------------|--------------------------------------|-------------------------------|
| Key used to verify/open | Sender's <b>public</b> key           | Receiver's <b>private</b> key |
| Is the message hidden?  | <b>No</b> — signing does not conceal | <b>Yes</b>                    |

**Digital certificate and PKI:** a public key alone proves nothing — how do you know it belongs to your bank and not to an attacker? A **Certificate Authority (CA)** issues a **digital certificate (X.509)** binding an identity to a public key and **signs it with the CA's own private key**. Your browser trusts a set of root CAs, so it can verify the chain. That whole apparatus — CAs, certificates, registration authorities, revocation lists (CRL/OCSP) — is the **Public Key Infrastructure (PKI)**.

## Firewalls

A **firewall** is a hardware or software barrier placed between a trusted internal network and an untrusted external one, which **inspects traffic and permits or denies it according to a rule set (ACL)**. It enforces the network's security policy at a single choke point.

| Type                                     | Layer      | How it works                                                                                                                                 | Strengths                                                                 | Weaknesses                                                                                                                                |
|------------------------------------------|------------|----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Packet filter</b>                     | <b>3–4</b> | Examines each packet's <b>source/destination IP, port, protocol and flags</b> against static rules; <b>stateless</b>                         | <b>Fast</b> , cheap, transparent                                          | <b>No context</b> — cannot tell a reply from an unsolicited packet; cannot see payloads; vulnerable to spoofing and fragmentation attacks |
| <b>Stateful inspection</b>               | <b>3–4</b> | Maintains a <b>state table of active connections</b> ; permits inbound packets only if they belong to a connection initiated inside          | Much stronger than static filtering; still fast                           | Cannot inspect application content; state table is a DoS target                                                                           |
| <b>Application-level gateway / proxy</b> | <b>7</b>   | A <b>proxy</b> terminates the connection and re-originates it, understanding the specific protocol (HTTP, FTP) and inspecting <b>content</b> | <b>Deep inspection</b> , logging, content filtering, hides internal hosts | <b>Slow</b> ; a separate proxy is needed per application                                                                                  |
| <b>Circuit-level gateway</b>             | <b>5</b>   | Validates the TCP handshake, then relays bytes without inspecting them (SOCKS)                                                               | Low overhead, hides internal addresses                                    | No content inspection                                                                                                                     |
| <b>NGFW / UTM</b>                        | <b>3–7</b> | Combines stateful inspection with IPS, deep packet inspection, application awareness, TLS inspection                                         | Comprehensive                                                             | Costly, complex, can be a bottleneck                                                                                                      |

**DMZ (Demilitarised Zone)** — a subnet between two firewalls (or on a third interface of one) holding **public-facing servers** (web, mail, DNS). If a DMZ server is compromised, the attacker is still separated from the internal LAN by another firewall. Draw this: Internet → outer firewall → DMZ (web/mail servers) → inner firewall → internal LAN.

**What a firewall cannot do** — worth two marks: it cannot stop attacks that **bypass** it (rogue modems, USB drives, personal hotspots), it cannot stop **insider** attacks, it cannot detect malware in **encrypted** traffic without TLS interception, and it cannot patch a vulnerable application. Complement with **IDS/IPS**, **antivirus** and patching.

**IDS vs IPS**: an **Intrusion Detection System** monitors and **alerts** (passive, out of band); an **Intrusion Prevention System** sits **in line** and **blocks**. Detection methods: **signature-based** (matches known patterns; misses zero-days) and **anomaly/behaviour-based** (flags deviations from a baseline; more false positives).

**VPN**: a **Virtual Private Network** creates an **encrypted tunnel** across a public network, so a remote user or branch office behaves as if directly attached to the private LAN. Protocols: **IPSec** (**AH** for authentication/integrity, **ESP** for encryption; **transport mode** protects the payload, **tunnel mode** encapsulates the entire original packet in a new one), **SSL/TLS VPN**, PPTP, L2TP, WireGuard.

## Tunnelling, fragmentation and internetworking

These three are named explicitly in your Forensic syllabus, so they need clean definitions.

**Tunnelling** is **encapsulating one protocol's packet inside another protocol's packet** so that it can traverse a network that does not support it. The classic example: two IPv6 islands separated by an IPv4-only Internet — each IPv6 packet is placed inside an IPv4 packet at the entry router, carried across as ordinary IPv4 payload, and unwrapped at the exit router. The intermediate network never knows IPv6 exists. Applications: **6to4 / Teredo** (IPv6 over IPv4), **VPNs** (IPSec tunnel mode, L2TP, PPTP, GRE), **MPLS**, connecting two remote LANs of the same type over a WAN of a different type. Analogy that works well in an answer: putting a car on a train to cross a mountain — the car doesn't drive through, it is *carried*.

**Fragmentation** happens because different networks have different **MTUs** (Maximum Transmission Unit — the largest payload a link can carry; **Ethernet's MTU is 1500 bytes**). When a router must forward a packet onto a link with a smaller MTU, it **breaks the packet into fragments**, each with its own IP header.

- IPv4 header fields used: **Identification** (all fragments of one packet share it), **Flags** — **DF** (Don't Fragment) and **MF** (More Fragments; set on all but the last), and **Fragment Offset** (position of this fragment's data, **measured in 8-byte units**).
- **Reassembly is done only at the final destination**, never at intermediate routers, because fragments may take different routes.
- **Costs**: more headers (overhead), more processing, and **loss of any one fragment forces retransmission of the whole original packet**.

- **Transparent fragmentation** — the fragments are reassembled at the exit gateway of the small-MTU network, so the next network sees the original packet.
- **Non-transparent fragmentation** — fragments travel independently all the way to the destination host, which reassembles. **IP uses non-transparent fragmentation.**
- **IPv6 abolishes router fragmentation:** only the **source** may fragment, and it uses **Path MTU Discovery** (send with DF set; a router that cannot forward returns ICMP "fragmentation needed", revealing the path MTU).
- **Forensic relevance:** overlapping and tiny fragments are a classic technique for **evading IDS/firewalls** (teardrop attack, fragment overlap).

**Worked fragmentation example.** A 4000-byte IP packet (20-byte header + 3980 bytes of data) must cross a link with **MTU 1500**.

- Each fragment may carry at most  $1500 - 20 = 1480$  bytes of data, and the data length of every fragment except the last must be a **multiple of 8**. 1480 is a multiple of 8 ✓
- Fragment 1: data bytes 0–1479, **offset = 0**, MF = 1, total length 1500
- Fragment 2: data bytes 1480–2959, **offset =  $1480/8 = 185$** , MF = 1, total length 1500
- Fragment 3: data bytes 2960–3979 (1020 bytes), **offset =  $2960/8 = 370$** , **MF = 0**, total length 1040
- Total: **3 fragments**; overhead rose from 20 bytes of header to 60.

## Likely exam questions

- **[5]** Differentiate between active and passive attacks with examples.
- **[5]** Differentiate between symmetric and asymmetric cryptography.
- **[5]** What is a digital signature? How does it provide non-repudiation?
- **[5]** What is a firewall? Explain any two types.
- **[5]** Differentiate between a virus, a worm and a Trojan horse.
- **[5]** What is tunnelling? Give one application.
- **[5]** Explain fragmentation. What is an MTU?
- **[10]** Explain public key cryptography. How is it used for confidentiality and for authentication? Describe the working of RSA and of digital signatures with a diagram.
- **[10]** Explain firewalls: types, working, placement, the DMZ, and their limitations.
- **[15]** Discuss network security in detail: security goals, active and passive attacks, symmetric and asymmetric cryptography with a comparison, hash functions, digital signatures and certificates, firewalls and VPNs.

## MCQ traps

| Trap                                           | Correct answer                    |
|------------------------------------------------|-----------------------------------|
| "For confidentiality, encrypt with whose key?" | The <b>receiver's public</b> key. |

|                                                    |                                                                         |
|----------------------------------------------------|-------------------------------------------------------------------------|
| "For a digital signature, encrypt with whose key?" | The <b>sender's private</b> key.                                        |
| "Which provides non-repudiation?"                  | <b>Asymmetric / digital signature</b> , never symmetric.                |
| "Keys needed for n users, symmetric?"              | $n(n-1)/2$ . Asymmetric = <b>2n</b> .                                   |
| "DES key size?"                                    | <b>56 bits</b> (64 with parity). AES = 128/192/256.                     |
| "MD5 / SHA-1 digest size?"                         | <b>128 / 160</b> bits.                                                  |
| "RSA's hard problem?"                              | <b>Factoring large numbers</b> . Diffie–Hellman → <b>discrete log</b> . |
| "Diffie–Hellman encrypts data?"                    | <b>No</b> — it is <b>key exchange</b> only.                             |
| "Does signing hide the message?"                   | <b>No</b> .                                                             |
| "Which firewall type inspects content?"            | <b>Application-level gateway (proxy)</b> , layer 7.                     |
| "Ethernet MTU?"                                    | <b>1500 bytes</b> .                                                     |
| "Fragment offset is measured in?"                  | <b>8-byte units</b> .                                                   |
| "Who reassembles IP fragments?"                    | The <b>final destination</b> only.                                      |
| "Virus vs worm?"                                   | A <b>worm needs no host file and self-propagates</b> .                  |

## 19. ISDN, ATM and other WAN technologies

### ISDN — Integrated Services Digital Network

**The idea:** the telephone local loop was analogue, so every data transfer needed a modem and suffered analogue impairments. ISDN carries **voice, data, video and fax digitally over the existing copper telephone line**, integrating all services on one network.

**Channel types:**

| Channel           | Rate                                 | Purpose                                                                       |
|-------------------|--------------------------------------|-------------------------------------------------------------------------------|
| <b>B (bearer)</b> | <b>64 kbps</b>                       | Carries <b>user data</b> — voice or data                                      |
| <b>D (delta)</b>  | <b>16 kbps (BRI) / 64 kbps (PRI)</b> | Carries <b>signalling and control</b> (out-of-band), and low-rate packet data |
| H channels        | 384 kbps – 1.92 Mbps                 | High-rate user data                                                           |

**Interfaces — the numbers to memorise:**

| Interface                                                       | Composition                       | Total user data | Total bit rate              | Use                |
|-----------------------------------------------------------------|-----------------------------------|-----------------|-----------------------------|--------------------|
| <b>BRI</b> (Basic Rate Interface)                               | <b>2B + 1D(16k)</b>               | <b>128 kbps</b> | 144 kbps (192 with framing) | Home, small office |
| <b>PRI</b> (Primary Rate Interface), <b>North America/Japan</b> | <b>23B + 1D(64k)</b>              | 1.472 Mbps      | <b>1.544 Mbps (T1)</b>      | Business, PBX      |
| <b>PRI, Europe/India</b>                                        | <b>30B + 1D(64k)</b> (+1 framing) | 1.92 Mbps       | <b>2.048 Mbps (E1)</b>      | Business, PBX      |

**Narrowband ISDN (N-ISDN)** is the above — circuit-switched,  $\leq 2$  Mbps, over copper.

**Broadband ISDN (B-ISDN)** was the successor concept: rates **above 2 Mbps** (155 Mbps and up) over **optical fibre**, supporting video on demand and HDTV, and — this is the connection to make — **B-ISDN is implemented using ATM as its switching and multiplexing technology.**

## ATM — Asynchronous Transfer Mode

**The design problem ATM solves:** voice needs constant, low-jitter delivery (favouring circuit switching); data is bursty (favouring packet switching). ATM's answer is **cell relay** — packet switching with **small, fixed-size cells** over **virtual circuits**, giving both statistical efficiency *and* predictable delay.

| Property         | Value                                                                                                                                                 |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cell size</b> | <b>53 bytes = 5-byte header + 48-byte payload</b> — fixed                                                                                             |
| Switching        | <b>Connection-oriented, virtual circuits</b> (VPI/VCI in the header)                                                                                  |
| Multiplexing     | <b>Statistical (asynchronous) TDM</b> — slots are not pre-assigned                                                                                    |
| Speeds           | 155 Mbps (OC-3), 622 Mbps (OC-12), 2.5 Gbps and up                                                                                                    |
| Layers           | <b>Physical layer, ATM layer, AAL (ATM Adaptation Layer)</b> — AAL1 (CBR voice/video), AAL2 (VBR), AAL3/4, <b>AAL5</b> (data, "SEAL", the common one) |
| Service classes  | <b>CBR</b> (Constant Bit Rate), <b>VBR</b> (Variable), <b>ABR</b> (Available), <b>UBR</b> (Unspecified)                                               |

**Why fixed, small cells?** Two reasons, and giving both is the mark:

1. **Fixed size makes switching simple and fast enough to implement in hardware** — no variable-length parsing, predictable buffer management.
2. **Small size bounds the queueing delay.** A short voice cell never gets stuck behind a 9000-byte data frame, so **jitter stays low** — essential for real-time traffic.

The 48-byte payload was a political compromise between the European preference for 32 bytes (short enough to avoid echo cancellers) and the American preference for 64 (more efficient).

**Cell overhead:**  $5/53 \approx 9.4\%$  — the "cell tax", ATM's main criticism, alongside its complexity. ATM has largely been displaced by **Gigabit Ethernet**, **MPLS** and **IP/DWDM** in modern backbones, but it remains firmly on your syllabus.

## Other WAN technologies worth a line

| Technology         | Note                                                                                                                                                                                                                                                         |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>X.25</b>        | The original packet-switched WAN standard; connection-oriented virtual circuits, <b>heavy error checking at every hop</b> (designed for noisy analogue lines), hence slow ( $\leq 64$ kbps)                                                                  |
| <b>Frame Relay</b> | X.25's successor: virtual circuits identified by <b>DLCI</b> , <b>no per-hop error correction</b> (leaves it to the endpoints, since lines are now clean fibre) → much faster (up to 45 Mbps). Uses <b>CIR</b> (Committed Information Rate)                  |
| <b>SONET/SDH</b>   | Optical TDM hierarchy: <b>OC-1 = 51.84 Mbps</b> , OC-3 = 155.52 Mbps, OC-12, OC-48, OC-192. Ring topology with automatic protection switching                                                                                                                |
| <b>MPLS</b>        | Multi-Protocol Label Switching — forwards on short <b>labels</b> instead of IP lookups; "layer 2.5". Enables traffic engineering and VPNs                                                                                                                    |
| <b>Leased line</b> | A dedicated, always-on, symmetric point-to-point circuit rented from a carrier                                                                                                                                                                               |
| <b>PPP</b>         | Point-to-Point Protocol — the data link protocol for dial-up/serial WAN links; provides framing, <b>LCP</b> (link control), <b>NCP</b> (network control) and authentication ( <b>PAP</b> plaintext, <b>CHAP</b> challenge–response, which is the secure one) |

## MCQ traps

| Trap                            | Correct answer                                                      |
|---------------------------------|---------------------------------------------------------------------|
| "ATM cell size?"                | <b>53 bytes</b> (5 header + 48 payload).                            |
| "BRI composition and rate?"     | <b>2B + D, 128 kbps</b> of user data (144 kbps total).              |
| "PRI in Europe/India?"          | <b>30B + D = 2.048 Mbps (E1)</b> . In the US, 23B + D = 1.544 Mbps. |
| "ISDN B channel rate?"          | <b>64 kbps</b> .                                                    |
| "ATM is connection-oriented?"   | <b>Yes</b> — virtual circuits.                                      |
| "B-ISDN uses which technology?" | <b>ATM</b> .                                                        |

|                        |                                                                                              |
|------------------------|----------------------------------------------------------------------------------------------|
| "Frame Relay vs X.25?" | Frame Relay <b>omits per-hop error correction</b> → faster.                                  |
| "CHAP vs PAP?"         | <b>CHAP</b> is the secure, challenge-based one; PAP sends the password in <b>plaintext</b> . |

## 20. IEEE standards and Ethernet

| Standard     | Covers                                                    |
|--------------|-----------------------------------------------------------|
| 802.1        | Bridging, spanning tree, <b>VLANs (802.1Q)</b>            |
| 802.2        | <b>LLC</b> (Logical Link Control) sub-layer               |
| 802.3        | <b>Ethernet / CSMA-CD</b>                                 |
| 802.4        | Token Bus                                                 |
| 802.5        | <b>Token Ring</b>                                         |
| 802.6        | <b>MAN — DQDB</b>                                         |
| 802.11       | <b>Wireless LAN — Wi-Fi</b> (a/b/g/n/ac/ax)               |
| 802.15       | <b>WPAN</b> — 802.15.1 <b>Bluetooth</b> , 802.15.4 Zigbee |
| 802.16       | <b>WiMAX</b> (broadband wireless MAN)                     |
| 802.3af / at | Power over Ethernet                                       |

### Wi-Fi generations:

| Standard           | Band        | Max rate        |
|--------------------|-------------|-----------------|
| 802.11a            | 5 GHz       | 54 Mbps         |
| 802.11b            | 2.4 GHz     | 11 Mbps         |
| 802.11g            | 2.4 GHz     | 54 Mbps         |
| 802.11n (Wi-Fi 4)  | 2.4/5 GHz   | 600 Mbps (MIMO) |
| 802.11ac (Wi-Fi 5) | 5 GHz       | ~3.5 Gbps       |
| 802.11ax (Wi-Fi 6) | 2.4/5/6 GHz | ~9.6 Gbps       |

Wireless security, in order of strength: **WEP (broken — RC4, weak IV) < WPA (TKIP) <**

**WPA2 (AES-CCMP) < WPA3 (SAE).**

**Ethernet frame format** (worth a diagram):

| Field                       | Size                                              |
|-----------------------------|---------------------------------------------------|
| Preamble                    | 7 bytes (alternating 1010... for synchronisation) |
| SFD (Start Frame Delimiter) | 1 byte (10101011)                                 |
| <b>Destination MAC</b>      | <b>6 bytes</b>                                    |
| <b>Source MAC</b>           | <b>6 bytes</b>                                    |
| Type/Length                 | 2 bytes                                           |
| <b>Data (payload)</b>       | <b>46–1500 bytes</b> (padded to 46 if shorter)    |
| <b>FCS (CRC)</b>            | <b>4 bytes</b>                                    |

Minimum frame size **64 bytes**, maximum **1518 bytes** (excluding preamble). The 64-byte minimum exists so that **collision detection works**: a station must still be transmitting when the collision signal returns from the far end of the maximum-length cable.

**CSMA/CD algorithm** (Ethernet): **listen** (carrier sense) → if idle, **transmit** → keep **listening while transmitting**; on collision, send a **32-bit jam signal**, then wait a random time chosen by **binary exponential backoff** (after the  $n$ th collision, wait a random number of slot times in  $[0, 2^n - 1]$ , capped at  $n = 10$ , abort after 16) and retry.

**CSMA/CA** (Wi-Fi) avoids collisions instead of detecting them, because a radio cannot listen while transmitting: it waits for an idle channel plus a random backoff, and optionally uses **RTS/CTS** handshaking to counter the **hidden terminal problem** (two stations that can both hear the AP but not each other).

**Protocol suites named in the Diploma syllabus:**

| Suite            | Origin         | Note                                                                                            |
|------------------|----------------|-------------------------------------------------------------------------------------------------|
| <b>TCP/IP</b>    | DARPA          | <b>Routable</b> , open, the Internet standard                                                   |
| <b>NetBEUI</b>   | IBM/Microsoft  | Small-LAN protocol, fast and simple, but <b>non-routable</b> — this is the exam point. Obsolete |
| <b>IPX/SPX</b>   | Novell NetWare | <b>Routable</b> ; IPX $\approx$ IP (network layer), SPX $\approx$ TCP (transport). Obsolete     |
| <b>AppleTalk</b> | Apple          | Legacy Mac networking                                                                           |

---

## 21. Last-week revision sheet

---

### Twenty facts most likely to be one-mark MCQs:

1. OSI has **7 layers (ISO, 1984)**; TCP/IP has **4** and lacks **Session and Presentation**.
2. PDUs: **bits** → **frame** → **packet** → **segment** → **data**.
3. **Encryption and compression = Presentation (6)**; checkpoints/dialogue = **Session (5)**.
4. **Router = layer 3, switch/bridge = layer 2, hub/repeater = layer 1, gateway = 7**.
5. **A switch breaks collision domains; a router breaks broadcast domains**.
6. Mesh links =  $n(n-1)/2$ .
7. Class ranges: **A 1–126, B 128–191, C 192–223, D 224–239, E 240–255**; 127 = loopback.
8. Private: **10.0.0.0/8, 172.16–172.31, 192.168.0.0/16**.
9. **/26 → 255.255.255.192, block 64, 62 hosts**. /27 → 224/32/30. /28 → 240/16/14. /30 → 252/4/2.
10. **MAC = 48 bits; IPv4 = 32 bits; IPv6 = 128 bits**.
11. **ARP maps IP → MAC**; RARP the reverse; DHCP = **DORA**, ports 67/68.
12. **RIP: hop count, max 15, 30 s, Bellman–Ford**. **OSPF: cost, Dijkstra, area 0**. **BGP: path vector, TCP 179**.
13. TCP header **20 bytes**, UDP **8 bytes**; setup **3-way**, teardown **4-way**.
14. Ports: **FTP 20/21, SSH 22, Telnet 23, SMTP 25, DNS 53, HTTP 80, POP3 110, IMAP 143, HTTPS 443**.
15. **Nyquist C = 2B log<sub>2</sub>L** (noiseless); **Shannon C = B log<sub>2</sub>(1 + S/N)** (noisy); **30 dB = S/N of 1000**.
16. **T1 = 1.544 Mbps, 24 channels; E1 = 2.048 Mbps, 32; DS0 = 64 kbps**.
17. **ATM cell = 53 bytes (5 + 48)**; **ISDN BRI = 2B + D = 128 kbps**.
18. **Confidentiality → receiver's public key. Signature → sender's private key**.
19. **MD5 = 128 bits, SHA-1 = 160, AES = 128/192/256, DES = 56**.
20. **Ethernet MTU 1500**, min frame **64 bytes**, max **1518**; **CSMA/CD** wired, **CSMA/CA** wireless.

**Descriptive-paper strategy.** Networks will supply Section A questions in all three papers, and it is the topic most likely to yield a **15-mark question you can fully answer**. Prepare these six as guaranteed long answers, each with its diagram:

- (a) **OSI model, all 7 layers with PDUs and encapsulation diagram**;
- (b) **transmission media with comparison table**;
- (c) **topologies with diagrams and advantages/disadvantages**;
- (d) **IP addressing with a full VLSM worked design** — the highest-scoring, because worked numericals are unambiguous to mark;
- (e) **TCP vs UDP with the three-way handshake diagram**;
- (f) **cryptography and digital signatures with the signing diagram**.

Always draw the diagram *first*, label it, and then write the prose around it — if you run out



of time, a labelled diagram still scores.